

The `ai-agent-terraform` deployment as pure Lean 4

Machine-checked properties, and a zero-knowledge proof of possession for the whole project

Abstract

This project replaces the HCL, shell scripts and systemd definitions of the `meta-introspector/ai-agent-terraform` repository — all 20 branches, 812 files, 63093 generated lines — with pure Lean 4 data and total renderers, and generates the original artifacts as side effects. On top of that description sits a zero-knowledge layer: every step of the project states a claim about itself, the claims are rolled up into one Pedersen commitment, and the project proves, in zero knowledge, that it possesses the data behind that commitment. This document collects the mathematics: the commitment scheme, the Σ -protocol and its three security properties, the roll-up and the Merkle openings, and the statements that hold of *this* project. Every theorem quoted here is proved in Lean and checked by the Lean kernel; the sources contain no `sorry` and no added axioms. Appendix A lists all 276 theorems of the project with their statements as written in the sources.

Contents

1	What is being proved about	2
2	The description and its properties	3
2.1	Rendering is faithful	3
2.2	The import is complete	3
2.3	The typed model	3
3	The zero-knowledge machinery	4
3.1	The field and the digest	4
3.2	Pedersen commitments	4
3.3	The Σ -protocol	4
3.4	Claims and roll-ups	5
3.5	Opening one step: Merkle trees	6
3.6	Signatures on claims	6
4	The proof of possession, applied to the project	7
4.1	What each step claims	7
4.2	The roll-up of the whole project	7
4.3	What the project proves in zero knowledge	8
4.4	The demonstration instance	8
5	What the machine checks, and how	9
6	Two honest caveats	9
A	Catalogue of the theorems	9
A.1	<code>AiAgentTerraform/Iface/Nix.lean</code>	9
A.2	<code>AiAgentTerraform/Ops.lean</code>	10
A.3	<code>AiAgentTerraform/Properties.lean</code>	11
A.4	<code>AiAgentTerraform/Model/Aws/Compute.lean</code>	13

A.5	<code>AiAgentTerraform/Model/Aws/Data.lean</code>	13
A.6	<code>AiAgentTerraform/Model/Aws/Iam.lean</code>	14
A.7	<code>AiAgentTerraform/Model/Aws/Modules.lean</code>	14
A.8	<code>AiAgentTerraform/Model/Aws/Network.lean</code>	15
A.9	<code>AiAgentTerraform/Model/Aws/Observability.lean</code>	15
A.10	<code>AiAgentTerraform/Model/Aws/Serverless.lean</code>	15
A.11	<code>AiAgentTerraform/Model/Aws/Ssm.lean</code>	16
A.12	<code>AiAgentTerraform/Model/Aws/Storage.lean</code>	16
A.13	<code>AiAgentTerraform/Model/Introspector.lean</code>	17
A.14	<code>AiAgentTerraform/Model/Packer.lean</code>	17
A.15	<code>AiAgentTerraform/Model/Properties.lean</code>	17
A.16	<code>AiAgentTerraform/Model/Resource.lean</code>	22
A.17	<code>AiAgentTerraform/Zkp/Bridge.lean</code>	22
A.18	<code>AiAgentTerraform/Zkp/Claim.lean</code>	23
A.19	<code>AiAgentTerraform/Zkp/Commit.lean</code>	25
A.20	<code>AiAgentTerraform/Zkp/Field.lean</code>	26
A.21	<code>AiAgentTerraform/Zkp/Merkle.lean</code>	26
A.22	<code>AiAgentTerraform/Zkp/Open.lean</code>	27
A.23	<code>AiAgentTerraform/Zkp/Project.lean</code>	28
A.24	<code>AiAgentTerraform/Zkp/Sigma.lean</code>	32
A.25	<code>AiAgentTerraform/Zkp/Sign.lean</code>	33
A.26	<code>AiAgentTerraform/Zkp/System.lean</code>	34

1 What is being proved about

The imported repository describes an AWS deployment of an AI agent: Terraform and OpenTofu configuration, Packer image templates, first-boot shell scripts, operator scripts, and the `systemctl` calls that install and start the agent’s service. In this project none of that is text. It is Lean data:

- `Hcl.Value` and `Hcl.Item` are HCL as inductive types — strings, numbers, booleans, expressions, tuples, objects, heredocs; attributes, blocks, comments and blank lines — with a total renderer `Hcl.render`.
- `Shell.Script` is a script as a list of typed statements (assignments, simple and continued commands, function definitions, loops, systemd actions, unit installation), and `Systemd.Unit` is a unit file as sections of key/value pairs; `Systemd.Action` is an interaction with the service manager.
- `GenFile` pairs a path with the artifact it is generated from, and `Branch` is a branch of the upstream repository as the list of files it generates. `allBranches` is the whole repository as one Lean value.
- `Tf.fullSystem` is the deployment as a *typed model*: resources and data sources with schemas, module calls, stacks, and the files the model renders to. `Ops.operations` are the operator scripts as typed values.
- `Emit` is the only effectful module: it writes files, runs scripts, drives `systemctl` and invokes `tofu`.

The driver reports ok: 12 stacks, 60 objects, no problems for the model, and the import covers 812 files across 20 branches.

This document is organised bottom-up: Section 2 states what is proved about the description itself, Section 3 develops the zero-knowledge machinery in the generality in which it is proved, and Section 4 applies it to the project. Throughout, the name in typewriter font after a statement is the Lean name of the theorem; its exact statement can be found under that name in Appendix A.

2 The description and its properties

2.1 Rendering is faithful

Theorem 2.1 (`Hcl.unescape_escape`). *For every string s , $\text{unescape}(\text{escape } s) = s$.*

Proof. Escaping maps a string to the concatenation, over its characters, of `escapeChar`, which is a two-character sequence `\c'` for the five characters that need escaping and the singleton `[c]` otherwise. Unescaping is the fold `unescapeAux`, which consumes a backslash together with the character after it and any other character alone. The proof is by induction on the character list. In the five escaped cases the head of the escaped list is a backslash followed by the escape letter, so `unescapeAux_escaped` rewrites it to `unescapeChar` of that letter — which is the original character — followed by the unescaping of the rest, and the induction hypothesis finishes. In the remaining case the head is the character itself, which is not a backslash, so `unescapeAux_cons_of_ne` applies. The statement about strings follows because `escape` and `unescape` are the character-list functions transported along `String.toList`. $\square$

Theorem 2.2 (`Systemd.sections_simpleService`). *Every unit built by `simpleService` has exactly the sections `[Unit]`, `[Service]`, `[Install]`, in that order.*

Proof. `simpleService` is a constructor application, so the list of section names reduces to the literal list; the two sides are definitionally equal. $\square$

2.2 The import is complete

Theorem 2.3 (`allBranches_length`, `allBranches_names`, `allBranches_file_count`, `paths_nonempty`). *The Lean description carries 20 branches, under the names listed in the theorem, generating 812 files counted with multiplicity; every branch generates at least one file and no generated path is empty.*

Proof. The first three are equalities between closed terms and are decided by reduction. The last is a decidable proposition over a finite list and is decided by evaluation of the deciding instance. $\square$

Theorem 2.4 (`operations_cover_imported_scripts`). *Every shell script of the imported repository is generated by one of the eight modelled operations: if f is a file of a branch and $f.\text{isShell}$, then $f.\text{path}$ occurs among the paths of `Ops.operations`.*

Proof. Again a decidable statement over finite data, decided by evaluation. $\square$

2.3 The typed model

A resource of the model is an address, a type name and a body of typed expressions; a schema for a type lists its arguments, which are required or optional, and its permitted blocks. `ResourceSchema.conforms` checks a resource against a schema and `errors` produces the diagnostics. For each modelled type there is a theorem that the values of that type conform to its schema, and `conforms_of_errors_nil` together with its converse identifies the boolean check with the absence of diagnostics.

Theorem 2.5 (`fullSystem_problems`). *The modelled system has no problems: no duplicate addresses, no dangling references, no schema violations, and no resource whose type is unknown to the registry. Every generated file has a distinct, non-empty path.*

Proof. The system, the registry and the checker are closed terms of decidable type; the statement is decided by evaluation. The individual conformance theorems are what make the same fact available per resource, which is what the claims of Section 3 use. $\square$

3 The zero-knowledge machinery

Everything in this section is proved for an arbitrary prime q and an arbitrary module M over $\mathbb{Z}/q$ — which is exactly a cyclic group of prime order q written additively — and for arbitrary public parameters. The concrete instantiation, in which M is the field itself so that the driver can compute, is discussed in Section 6.

3.1 The field and the digest

The field is $\mathbb{F}_p$ with $p = 2^{31} - 1$, a Mersenne prime (`modulus_prime`). Data are mapped into it by a deterministic arithmetic fold: $\text{step}(a, x) = a \cdot \beta + x + 1$ with $\beta = 1000003$, folded left over a list of field elements (`digestFp`) or over the code points of a string (`digestString`). Two facts about the fold are proved and used: $\text{step}(a, \cdot)$ is injective for each a (`digestStep_injective`) and $\text{step}(\cdot, x)$ is injective for each x (`digestStep_injective_acc`), the latter because $\beta \neq 0$ in a field. The fold is not a cryptographic hash, and no theorem in the project assumes that it is: where collision resistance would be needed — in the soundness of a Merkle opening — injectivity of the compression function is an explicit hypothesis of the theorem.

3.2 Pedersen commitments

Definition 3.1 (`Params.commit`). For public parameters $P = (g, h)$ in M , the commitment to m with randomness r is $\text{commit}(m, r) = m \cdot g + r \cdot h$.

Theorem 3.2 (`Params.commit_add`, `Params.sum_commit`, `Params.sum_commit_map`). *Commitments are additively homomorphic: $\text{commit}(m_1, r_1) + \text{commit}(m_2, r_2) = \text{commit}(m_1 + m_2, r_1 + r_2)$, and hence the sum of a list of commitments is the commitment of the sums.*

Proof. Immediate from distributivity of the scalar action: $m_1g + r_1h + m_2g + r_2h = (m_1 + m_2)g + (r_1 + r_2)h$. The list versions follow by induction, the empty case being $\text{commit}(0, 0) = 0$. $\square$

Theorem 3.3 (`Params.commit_hiding`, `Params.hidingEquiv_bijective`). *Suppose $g = s \cdot h$. Then for all m_1, m_2, r ,*

$$\text{commit}(m_1, r) = \text{commit}(m_2, r + (m_1 - m_2)s),$$

and $r \mapsto r + (m_1 - m_2)s$ is a bijection of the randomness space. A commitment therefore has the same distribution whatever value it hides.

Proof. Substituting $g = s \cdot h$, the left side is $(m_1s + r) \cdot h$ and the right side is $(m_2s + r + (m_1 - m_2)s) \cdot h$; the two scalars are equal. The map is translation by a constant, hence a bijection. $\square$

Theorem 3.4 (`Params.commit_binding`, `Params.value_unique`). *If $\text{commit}(m_1, r_1) = \text{commit}(m_2, r_2)$ with $m_1 \neq m_2$, then*

$$g = \frac{r_2 - r_1}{m_1 - m_2} \cdot h.$$

Consequently, if no s with $g = s \cdot h$ exists, a commitment determines the value committed to.

Proof. Rearranging the hypothesis gives $(m_1 - m_2) \cdot g = (r_2 - r_1) \cdot h$. Since $m_1 - m_2 \neq 0$ and $\mathbb{Z}/q$ is a field, multiply both sides by $(m_1 - m_2)^{-1}$. The second statement is the contrapositive. $\square$

3.3 The Σ -protocol

The protocol proves possession of an opening (m, r) of a published commitment C :

$$\begin{array}{llll} \text{prover picks } a_1, a_2 & \longrightarrow t = a_1g + a_2h \longrightarrow & & \\ & \longleftarrow e \longleftarrow & \text{verifier's challenge} & \\ z_1 = a_1 + em, z_2 = a_2 + er & \longrightarrow (z_1, z_2) \longrightarrow & z_1g + z_2h \stackrel{?}{=} t + e \cdot C & \end{array}$$

Theorem 3.5 (Params.completeness). *For all m, r, a_1, a_2, e , the honest transcript is accepted against $C = \text{commit}(m, r)$.*

Proof. $z_1g + z_2h = (a_1 + em)g + (a_2 + er)h = (a_1g + a_2h) + e(mg + rh) = t + eC$. $\square$

Theorem 3.6 (Params.special_soundness). *Let T_1, T_2 be accepting transcripts for the same C with $T_1.t = T_2.t$ and $T_1.e \neq T_2.e$. Then*

$$\text{extract}(T_1, T_2) = \left(\frac{z_1 - z'_1}{e - e'}, \frac{z_2 - z'_2}{e - e'} \right)$$

is an opening of C .

Proof. Subtracting the two verification equations, whose announcements agree, gives $\text{commit}(z_1 - z'_1, z_2 - z'_2) = (e - e') \cdot C$. Because $e - e' \neq 0$, scaling by $(e - e')^{-1}$ and using $c \cdot \text{commit}(m, r) = \text{commit}(cm, cr)$ (Params.smul_commit) yields commit of the extracted pair $= C$. $\square$

Theorem 3.7 (Params.honest_verifier_zk, simulateEquiv_bijective, Params.simulate_verify). *Let $\text{sim}(C, e, z_1, z_2)$ be the transcript with announcement $\text{commit}(z_1, z_2) - e \cdot C$. It always verifies, and for every challenge e and witness (m, r) ,*

$$\text{prove}(m, r, a_1, a_2, e) = \text{sim}(\text{commit}(m, r), e, a_1 + em, a_2 + er),$$

where $(a_1, a_2) \mapsto (a_1 + em, a_2 + er)$ is a bijection of the randomness space. So honest transcripts and simulated transcripts are equidistributed and the proof reveals nothing beyond the truth of the statement.

Proof. The responses on both sides are equal by definition, and the announcements agree because $\text{commit}(a_1, a_2) = \text{commit}(a_1 + em, a_2 + er) - e \cdot \text{commit}(m, r)$, again by the homomorphism. The randomness map is a product of translations, hence bijective. $\square$

Theorem 3.8 (Params.verify_combine, Params.verify_combineList). *Accepting transcripts that answer the same challenge combine, componentwise, into an accepting transcript for the sum of their statements.*

Proof. Adding the verification equations and using the homomorphism on the left and $e \cdot C_1 + e \cdot C_2 = e \cdot (C_1 + C_2)$ on the right. The list version is this step iterated, the empty list giving $\text{commit}(0, 0) = 0$. $\square$

Remark 3.9 (Fiat–Shamir). `fsProve` and `fsVerify` derive the challenge from the statement and the announcement with a public function; `Params.fsProve_eq_prove` says a non-interactive proof is the interactive transcript at the derived challenge, so soundness and zero knowledge transfer verbatim, and `Params.fsProve_verify` is completeness in that form.

3.4 Claims and roll-ups

Definition 3.10 (Claim, Attested, Rollup). A *claim* records what one step of the project is: its kind (resource, data, module, file or operation), its address, its type name, the digest of the artifact it generates, a boolean saying it is type-correct, its cost in cents per month and its resource usage. `Claim.encode` digests all of that into one field element. An *attested* claim is a claim together with a blinding nonce, and a *roll-up* is a tree whose leaves are attested claims and whose nodes are named groups.

Definition 3.11 (Rollup.commitTo). For a quantity f on claims, $\text{commitTo}(P, f, t)$ is the sum, over the leaves of t , of $\text{commit}(f(\text{claim}), \text{nonce})$.

Theorem 3.12 (Rollup.commitTo_eq, Rollup.commitTo_node). *The commitment of a tree is a single Pedersen commitment, to the total of f over its leaves, blinded by the total of the nonces; and the commitment of a node is the sum of the commitments of its children.*

Proof. The first is `Params.sum_commit_map` applied to the list of leaves. The second is induction over the children, using associativity of the sum of a concatenation. $\square$

Theorem 3.13 (`Rollup.cost_commitment`, `Rollup.cost_binding`). *The commitment of a roll-up to the cost quantity opens to the cast of its total cost in cents; and if it also opens to a different value, the discrete logarithm of g in base h follows.*

Proof. The first combines `commitTo_eq` with `sum_costFp_eq`, which says that casting the natural-number total into the field is the total of the casts (induction on the list). The second is binding. $\square$

Theorem 3.14 (`Rollup.step_proofs_aggregate`, `Rollup.prove_verify`). *If every leaf proves possession of the opening of its own commitment against a common challenge, the combination of those proofs is accepted against the commitment of the whole tree; and the proof computed from the tree's own aggregate is accepted as well.*

Proof. Each leaf's transcript is accepted by completeness; `verify_combineList` combines them into an accepting transcript for the sum of the per-leaf commitments, which is `commitTo` by definition. The second statement is completeness of the non-interactive prover after rewriting the tree's commitment with `commitTo_eq`. $\square$

3.5 Opening one step: Merkle trees

The roll-up proves things about the aggregate; the Merkle layer lets a single step be shown to belong to the published project while the others stay behind their digests.

Theorem 3.15 (`exists_path`). *For every compression function H , every tree and every leaf value of it, there is an authentication path that recomputes the root from that value.*

Proof. Induction on the tree. At a leaf the empty path works. At a node, the value lies in one of the two subtrees; prepend to the path given by the induction hypothesis the flag saying which side it is and the root of the sibling. $\square$

Theorem 3.16 (`mem_leaves_of_rootWith`). *If H is injective, and a path of length n recomputes the root of a perfect tree of depth n from a value v , then v is a leaf of that tree.*

Proof. Induction on n . For $n = 0$ the path is empty and v is the leaf itself. For $n+1$ the tree is a node and the path is non-empty; the recomputation equation is H applied to the two sides, so injectivity gives that the recomputation from the tail of the path equals the root of the corresponding subtree, and the induction hypothesis places v among its leaves. $\square$

Theorem 3.17 (`rootWith_pathIn`, `length_pathIn`). *`pathIn` computes the authentication path of the i -th leaf of a perfect tree of depth d , for $i < 2^d$: it has length d and recomputes the root.*

Proof. Induction on d , splitting on whether $i < 2^{d-1}$; in each case the recomputation unfolds to H of the two subtree roots, one of which is given by the induction hypothesis. $\square$

3.6 Signatures on claims

`SigKey.sign` is Schnorr's signature in the same group. Correctness (`sign_verify`) is the same computation as completeness; key recovery from a reused nonce (`key_recovery`) and message binding (`message_binding`) are the same divisions as special soundness; and `sign_is_proof_of_knowledge` says a signature is a Σ -protocol transcript for possession of the secret key, so the extractor and the simulator apply to it unchanged. `signed_rollup_verify` lifts correctness to every claim of a roll-up.

4 The proof of possession, applied to the project

4.1 What each step claims

Each modelled resource or data source claims its address, its type, the digest of the configuration it renders to, its conformance to the schema of its type, and the cost and usage the published price list assigns to that type (`claimOfResource`); a module call claims the digest of the call (`claimOfModule`); a generated file claims its path and the digest of its content (`claimOfFile`); an operation claims the digest of its script and carries the conformance flag “has a shebang” (`claimOfOp`).

Theorem 4.1 (`system_claims_conform`, `conforms_of_errors_nil`). *The conformance flag a claim carries is the model’s own schema check: no claim can be marked type-correct that the model would reject.*

4.2 The roll-up of the whole project

The system roll-up `systemRollup` covers the 84 native steps: the 60 modelled objects, the 16 files the model generates and the 8 operations. The project roll-up adds the imported repository:

```
def branchRollup (seed : Fp) (b : Branch) : Rollup :=
  .node b.name (b.files.map (fun f => leafOf seed (claimOfFile f)))

def branchesRollup (seed : Fp) : Rollup :=
  .node "imported" (allBranches.map (branchRollup seed))

def projectRollup (seed : Fp) : Rollup :=
  .node "ai-agent-terraform" [systemRollup seed, branchesRollup seed]
```

Theorem 4.2 (`projectRollup_size`, `projectRollup_size_eq`). *The project rolls up exactly one step per native step and one per imported file:*

$$|projectRollup| = |systemRollup| + \sum_{b \in allBranches} |b.files| = 84 + 812 = 896.$$

Proof. The claims of a branch’s roll-up are the claims of its files (`branchRollup_claims`: mapping a leaf constructor and then taking leaves is mapping the claim function), so the claims of the branch roll-up are the concatenation over branches (`branchesRollup_claims`), and the claims of the project are the system’s followed by those (`projectRollup_claims_eq`). Taking lengths gives the displayed sum; the numerical form then follows from `systemRollup_size_eq` and `allBranches_file_count`, each of which is a closed computation. $\square$

Theorem 4.3 (`projectRollup_covers_system`, `projectRollup_covers_branch_files`, `projectRollup_covers_resources`, `projectRollup_covers_operations`). *Nothing is left out: every leaf of the system roll-up, every file of every imported branch, every modelled resource and every operation occurs as a step of the project roll-up.*

Proof. Membership in the leaves of a node is membership in the leaves of one of its children (`mem_leaves_node`); the four statements are that chain of memberships followed down the tree. $\square$

Theorem 4.4 (`projectRollup_allConform`). *Every one of the 896 steps of the project is type-correct.*

Proof. Splitting the claim list at the append, the native half is `systemRollup_allConform`. For the imported half, the conformance flag of a file claim is by definition `f.path != ""`, and `paths_nonempty` says no path of any branch is empty. $\square$

Theorem 4.5 (`projectRollup_costCents_eq`, `projectRollup_usage_eq`). *The project attests to 31.10 per month on 4 vCPUs, 8 GiB of memory and 60 GiB of storage across 6 instances — the figures of the modelled infrastructure.*

Proof. Cost and usage roll up over the children of a node (`Rollup.costCents_node`, `Rollup.usage_node`). A file claim has cost 0 and zero usage by definition, so the branch roll-up contributes nothing (`branchesRollup_costCents`, `branchesRollup_usage`, by induction over the claim list), and what remains is `systemRollup_costCents_eq` and `systemRollup_usage_eq`. $\square$

4.3 What the project proves in zero knowledge

Write $C_{\text{project}} = \text{commitTo}(P, \text{cost}, \text{projectRollup})$ for the single group element the project publishes.

Theorem 4.6 (`projectCommitment_opens`, `projectCommitment_splits`). *C_{project} is a single Pedersen commitment to the project’s total cost, blinded by the sum of the 896 nonces, and it is the sum of the commitment of the modelled system and the commitment of the imported branches.*

Theorem 4.7 (`project_zkpop_complete`, `project_step_proofs_aggregate`). *Completeness. The project always convinces the verifier that it possesses an opening of C_{project} ; and the per-step proofs, answering a common challenge, combine into exactly such an accepted proof.*

Theorem 4.8 (`project_zkpop_sound`, `project_zkpop_cost_is_declared`). *Soundness. Two accepting transcripts with the same announcement and different challenges yield an opening of C_{project} ; and if no discrete logarithm of g in base h exists, the value extracted is exactly the total the 896 steps declared.*

Theorem 4.9 (`project_zkpop_zero_knowledge`). *Zero knowledge. The proof the project publishes equals what the witness-free simulator produces from the image of the prover’s randomness under a bijection, so it discloses neither the individual claims nor the total.*

Proof of the three. Each is the corresponding general theorem of Section 3 instantiated at C_{project} , after rewriting it as a single commitment with `Rollup.commitTo_eq`; the identification of the extracted value is `Params.value_unique` applied to the two openings. $\square$

Theorem 4.10 (`project_step_opens`, `project_open_sound`, `projectOpen_verifies`). *The 896 step digests are hashed into a perfect tree of depth 10 (1024 leaves, the surplus padded with zeros) whose root `projectMerkleRoot` is published. Every step has an authentication path recomputing that root, the path is computed and not merely asserted to exist, and — assuming the compression function injective — a path of depth 10 accepted against the root can only have opened a digest the project really contains, or the padding.*

Proof. $896 \leq 2^{10}$, so the completeness and soundness theorems of the Merkle layer apply to the project’s roll-up, and `rootWith_pathIn` gives the computed opening. $\square$

Theorem 4.11 (`project_steps_signed`). *Signing the encodings of all 896 claims under one key produces signatures that all verify against the single public key.*

4.4 The demonstration instance

With the concrete parameters $g = 5$, $h = 7$ over $\mathbb{F}_p$ itself, the driver computes the following for seed 12345 and prover randomness (31337, 271828):

steps rolled up	896 (of which 828 generated files)
all type-correct	yes
declared cost	31.10 per month
commitment C	739633731
Merkle root (depth 10)	480077755
announcement t	2059481
challenge e	1061365092
responses (z_1, z_2)	(163102018, 1891648269)
verifier’s decision	yes

For comparison, the native system alone (84 steps) publishes the commitment 1559160 and the root 742613250 at the same seed.

Theorem 4.12 (`demoProjectVerify_true`). *For every seed and every choice of prover randomness, the project’s demonstration proof is accepted.*

Proof. Completeness of the non-interactive protocol, transported across the equivalence between the boolean verifier and the propositional one (`Params.verifyB_iff`). $\square$

5 What the machine checks, and how

All 276 theorems are checked by the Lean kernel when the project is built; the sources contain no `sorry`, no `axiom` declaration and no use of `native_decide`. Alongside the proofs, five scripts exercise the compiled driver against the world outside Lean:

<code>tools/check_fidelity.py</code>	regenerates all 812 imported files and compares them with the originals
<code>tools/check_model.py</code>	re-parses the generated configuration with an HCL parser
<code>tools/check_ops.py</code>	writes the operator scripts and checks them with <code>bash -n</code>
<code>tools/check_zkp.py</code>	proves, verifies, tampers and opens, over the 84 native steps
<code>tools/check_zkp_project.py</code>	the same over all 896 steps of the project

The last of these checks that the roll-up covers 896 type-correct steps over 20 branches, that the imported files leave the aggregate cost unchanged, that an honest proof is accepted while any single tampered component is rejected, that a proof made under one seed does not verify against another seed’s commitment, that commitment and root are reproducible from the seed, and that steps open against the published root while nothing beyond the last does.

6 Two honest caveats

The price list. `costTable` is a coarse published table of cents per month per resource type. The theorems say the published aggregate is the sum of those figures; they do not say those figures are what a provider bills.

The demonstration parameters. `demoParams` commits inside $\mathbb{F}_p$ itself, which keeps the layer computable so that `aat zkp` can produce and check a proof inside Lean. The discrete logarithm is easy there, so the security statements — binding, and the identification of the extracted value — are the general ones, proved for an arbitrary group of prime order and arbitrary parameters, and they are what a deployment over a group in which the discrete logarithm is hard would rely on. Likewise the driver derives its nonces from a seed so that runs are reproducible; the hiding theorem is about a uniformly chosen nonce, and it is the seed that has to be kept secret.

A Catalogue of the theorems

Every theorem and lemma of the Lean sources, grouped by the file it lives in and in source order, with its documentation and its statement exactly as written. Proofs are omitted here; they are in the sources and are checked by the kernel.

A.1 `AiAgentTerraform/Iface/Nix.lean`

`AiAgentTerraform.Iface.Nix.sizeOf_expr_pos`

Every Nix expression has positive size (used for the termination proofs of the renderer).

```
@[simp] theorem sizeOf_expr_pos (e : Expr) : 0 < sizeOf e
```

AiAgentTerraform.Iface.Nix.sizeOf_exprList_pos

Every list of Nix expressions has positive size.

```
@[simp] theorem sizeOf_exprList_pos (l : List Expr) : 0 < sizeOf l
```

A.2 AiAgentTerraform/Ops.lean

AiAgentTerraform.Ops.refreshCmd_render

The refresh operation rolls exactly the auto scaling group it is given.

```
theorem refreshCmd_render (asg : String) :  
  (refreshCmd asg).render =  
    "aws autoscaling start-instance-refresh" ++ " " ++  
    ("--auto-scaling-group-name " ++ asg)
```

AiAgentTerraform.Ops.refreshCmd_stmt

A one-option invocation stays on one line.

```
theorem refreshCmd_stmt (asg : String) :  
  (refreshCmd asg).stmt = .raw ((refreshCmd asg).render)
```

AiAgentTerraform.Ops.observability_systemd_actions

The CloudWatch installer drives the service manager exactly as the instance bootstrap does: it enables the agent, starts it, and checks it.

```
theorem observability_systemd_actions (p : String) :  
  systemdActions (observabilityInstall p).stmts =  
    [ .enable Deployment.cloudWatchAgent  
    , .start Deployment.cloudWatchAgent  
    , .status Deployment.cloudWatchAgent ]
```

AiAgentTerraform.Ops.observability_matches_bootstrap

The operator's installer and the first-boot bootstrap agree on what they ask of systemd.

```
theorem observability_matches_bootstrap (p : String) :  
  systemdActions (observabilityInstall p).stmts =  
    systemdActions (Deployment.cloudWatchSetup p)
```

AiAgentTerraform.Ops.observability_fetches_config

The installer fetches the agent configuration from the SSM parameter it is given.

```
theorem observability_fetches_config (p : String) :  
  Stmt.raw ("/opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent-ctl "  
    ++ "-a fetch-config -m ec2 -s -c \{"ssm:" ++ p ++ "\}"")  
  ∈ (observabilityInstall p).stmts
```

AiAgentTerraform.Ops.ops_parameter_is_model_parameter

That parameter is the one the modelled operations stack creates, and the one the instance bootstrap reads.

```
theorem ops_parameter_is_model_parameter :  
  Tf.Swarm.cloudwatchConfig.parameterName = Tf.Expr.str cwParameter
```

AiAgentTerraform.Ops.deployRun_settings

The deployment driver defaults every one of its eleven settings so that the environment can override it, and exports the three the SSM document reads.

```
theorem deployRun_settings (d : DeployRun) :  
  d.settings.length = 11 ∧  
  d.settings.filter (fun s => match s with | .assign _ _ e => e | _ => false) =  
    [ envDefault "REGION" d.region true  
    , envDefault "GIT_NAME" d.gitName true  
    , envDefault "GIT_VERSION" d.gitVersion true ]
```

AiAgentTerraform.Ops.deployRun_settings_overridable

Every variable of the run is defaulted with the "\${NAME:-...}" idiom, so a run can be retargeted from the environment without editing anything.

```
theorem deployRun_settings_overridable (d : DeployRun) :  
  ∀ s ∈ d.settings, ∃ n v e, s = Stmt.assign n ("\"${" ++ n ++ ":-" ++ v ++ "}\"") e
```

AiAgentTerraform.Ops.callSwarms_structure

The deployment driver defines the three functions it uses, and its main loop runs over the instances the first of them finds.

```
theorem callSwarms_structure (d : DeployRun) :
  (callSwarmsScript d).stmts.filterMap (fun s => match s with
  | .funcDef n _ => some n | _ => none) =
  ["get_instance_ids", "send_command", "fetch_command_output"] ^
  (callSwarmsScript d).stmts.filterMap (fun s => match s with
  | .forLoop v items _ => some (v, items) | _ => none) =
  [{"instance", "$(get_instance_ids)"]
```

AiAgentTerraform.Ops.callSwarms_send_command

The command it sends names the document and its version, bounds the run, and captures the output both to S3 and to CloudWatch Logs.

```
theorem callSwarms_send_command (d : DeployRun) :
  (DeployRun.sendCommand d).opts.map Prod.fst =
  [ "document-name", "document-version", "targets", "parameters"
  , "timeout-seconds", "max-concurrency", "max-errors"
  , "output-s3-bucket-name", "cloud-watch-output-config" ] ^
  (DeployRun.sendCommand d).region = some "$REGION"
```

AiAgentTerraform.Ops.restart_uses_model_document

The restart operation uses the SSM document the modelled operations stack creates, and asks after the unit the deployment installs.

```
theorem restart_uses_model_document :
  Tf.Swarm.restartDocument.documentName = Tf.Expr.str "swarms-restart-api" ^
  systemdActions (restartApiScript "swarms-restart-api").stmts =
  [ .status Deployment.swarmsApiUnit.fileName]
```

AiAgentTerraform.Ops.opsBranch_wellFormed

Every operation is a distinct, non-empty path, every script is written executable, and every script says how it is to be run.

```
theorem opsBranch_wellFormed :
  opsBranch.paths.Nodup ^
  opsBranch.files.all (fun f => !f.path.isEmpty) = true ^
  (operations.all (fun o => o.script.shebang == some "#!/bin/bash")) = true
```

AiAgentTerraform.Ops.operations_names

The operations cover the operational scripts of the imported repository: the auto scaling refresh, the CloudWatch installer, the SSM deployment driver and the three log tools — plus the API restart, which is new here.

```
theorem operations_names :
  operations.map Op.name =
  ["refresh", "refresh-eliza", "observability", "deploy", "restart"
  , "getlogs", "report", "analyse"]
```

A.3 AiAgentTerraform/Properties.lean

AiAgentTerraform.Hcl.unescapeAux_cons_of_ne

```
theorem unescapeAux_cons_of_ne {c : Char} (h : c ≠ '\{\}\}') (cs : List Char) :
  unescapeAux (c :: cs) = c :: unescapeAux cs
```

AiAgentTerraform.Hcl.unescapeAux_escaped

```
theorem unescapeAux_escaped (d : Char) (ds : List Char) :
  unescapeAux ('\{\}\{' :: d :: ds) = unescapeChar d :: unescapeAux ds
```

AiAgentTerraform.Hcl.unescapeAux_escapeChars

```
theorem unescapeAux_escapeChars (cs : List Char) : unescapeAux (escapeChars cs) = cs
```

AiAgentTerraform.Hcl.unescape_escape

Escaping a string for an HCL literal loses no information: the escaped form decodes back to the original string.

```
theorem unescape_escape (s : String) : unescape (escape s) = s
```

AiAgentTerraform.Systemd.sections_simpleService

A service built by `simpleService` has exactly the three sections of a well-formed unit file, in the canonical order.

```
theorem sections_simpleService (fileName description execStart : String)
  (wd : Option String) (user : String) (after : List String)
  (env : List (String × String)) (restart wantedBy : String) :
  ((simpleService fileName description execStart wd user after env restart
    wantedBy).sections.map Section.name) = ["Unit", "Service", "Install"]
```

AiAgentTerraform.Systemd.execStart_simpleService

The `ExecStart` of a service built by `simpleService` is the command it was given.

```
theorem execStart_simpleService (fileName description execStart : String)
  (user : String) (after : List String) (restart wantedBy : String) :
  ("ExecStart", execStart) ∈
  (simpleService fileName description execStart none user after [] restart
    wantedBy).sections[1]!.entries)
```

AiAgentTerraform.allBranches_length

All twenty branches of the upstream repository are present.

```
theorem allBranches_length : allBranches.length = 20
```

AiAgentTerraform.allBranches_names

The names of the imported branches.

```
theorem allBranches_names :
  allBranches.map Branch.name =
  ["feature/aider/step1", "feature/aitokenteam", "feature/alb", "feature/call_swarms",
   "feature/cloudformation", "feature/codebuild", "feature/cognito", "feature/docker",
   "feature/ec2", "feature/ec2-many-sizes", "feature/eliza.introspector.meme",
   "feature/github_secrets", "feature/godaddy", "feature/mcs", "feature/ses",
   "feature/swarmdeploy", "feature/swarms-ai-simple", "feature/swarms.ai",
   "kagenti-onboarding", "main"]
```

AiAgentTerraform.allBranches_file_count

The Lean description covers all 812 configuration files of the upstream repository (counted with multiplicity across branches).

```
theorem allBranches_file_count :
  (allBranches.map (fun b => b.files.length)).sum = 812
```

AiAgentTerraform.paths_nonempty

Every branch generates at least one file, and no path is empty.

```
theorem paths_nonempty : ∀ b ∈ allBranches, b.files ≠ [] ∧ ∀ p ∈ b.paths, p ≠ ""
```

AiAgentTerraform.operations_cover_imported_scripts

Every shell script of the imported repository is one of the operations modelled in `AiAgentTerraform/Ops.lean`: nothing upstream runs a `.sh` file that the typed operations layer does not generate.

```
theorem operations_cover_imported_scripts :
  ∀ b ∈ allBranches, ∀ f ∈ b.files,
  f.isShell → f.path ∈ Ops.operations.map Ops.Op.path
```

AiAgentTerraform.Deployment.bootstrap_enables_cloudwatch

The bootstrap script enables the CloudWatch agent through `systemd`.

```
theorem bootstrap_enables_cloudwatch (branch installScript ssm repo : String) :
  Shell.Stmt.systemd (.enable cloudWatchAgent) ∈
  (bootstrapScript branch installScript ssm repo).stmts
```

AiAgentTerraform.Deployment.bootstrap_installs_api_unit

The bootstrap script installs the API service unit and starts it.

```
theorem bootstrap_installs_api_unit (branch installScript ssm repo : String) :
  Shell.Stmt.installUnit swarmsApiUnit ∈
  (bootstrapScript branch installScript ssm repo).stmts ∧
  Shell.Stmt.systemd (.enableNow swarmsApiUnit.fileName) ∈
  (bootstrapScript branch installScript ssm repo).stmts
```

AiAgentTerraform.Deployment.launchTemplate_user_data

The launch template embeds exactly the bootstrap script as its user data.

```
theorem launchTemplate_user_data (name branch installScript ssm : String) :  
  Hcl.locals [Hcl.Item.attr "user_data" (bootstrapHeredoc branch installScript ssm)] ∈  
    launchTemplate name branch installScript ssm
```

A.4 AiAgentTerraform/Model/Aws/Compute.lean

AiAgentTerraform.Tf.Aws.LaunchTemplate.toResource_conforms

Every launch template model elaborates to a conforming resource, whichever of its optional attributes and blocks are set.

```
theorem toResource_conforms (t : LaunchTemplate) : schema.conforms t.toResource = true
```

AiAgentTerraform.Tf.Aws.Instance.toResource_conforms

Instances conform to their schema.

```
theorem toResource_conforms (i : Instance) : schema.conforms i.toResource = true
```

AiAgentTerraform.Tf.Aws.KeyPair.toResource_conforms

Key pairs conform to their schema.

```
theorem toResource_conforms (k : KeyPair) : schema.conforms k.toResource = true
```

A.5 AiAgentTerraform/Model/Aws/Data.lean

AiAgentTerraform.Tf.Aws.Data.AvailabilityZones.toResource_conforms

Availability-zone lookups conform to their schema.

```
theorem toResource_conforms (z : AvailabilityZones) : schema.conforms z.toResource = true
```

AiAgentTerraform.Tf.Aws.Data.CallerIdentity.toResource_conforms

Caller-identity lookups conform to their schema.

```
theorem toResource_conforms (c : CallerIdentity) : schema.conforms c.toResource = true
```

AiAgentTerraform.Tf.Aws.Data.S3Bucket.toResource_conforms

Bucket lookups conform to their schema.

```
theorem toResource_conforms (b : S3Bucket) : schema.conforms b.toResource = true
```

AiAgentTerraform.Tf.Aws.Data.SsmParameter.toResource_conforms

Parameter lookups conform to their schema.

```
theorem toResource_conforms (p : SsmParameter) : schema.conforms p.toResource = true
```

AiAgentTerraform.Tf.Aws.Data.IamUser.toResource_conforms

IAM user lookups conform to their schema.

```
theorem toResource_conforms (u : IamUser) : schema.conforms u.toResource = true
```

AiAgentTerraform.Tf.Aws.Data.CloudflareZone.toResource_conforms

Cloudflare zone lookups conform to their schema.

```
theorem toResource_conforms (z : CloudflareZone) : schema.conforms z.toResource = true
```

AiAgentTerraform.Tf.Aws.Data.Ami.toResource_conforms

Image lookups conform to their schema.

```
theorem toResource_conforms (a : Ami) : schema.conforms a.toResource = true
```

AiAgentTerraform.Tf.Aws.Data.CloudtrailServiceAccount.toResource_conforms

CloudTrail service-account lookups conform to their schema.

```
theorem toResource_conforms (c : CloudtrailServiceAccount) :  
  schema.conforms c.toResource = true
```

AiAgentTerraform.Tf.Aws.Data.DynamodbTable.toResource_conforms

Table lookups conform to their schema.

```
theorem toResource_conforms (t : DynamodbTable) : schema.conforms t.toResource = true
```

AiAgentTerraform.Tf.Aws.Data.IamPolicy.toResource_conforms

Managed-policy lookups conform to their schema.

```
theorem toResource_conforms (p : IamPolicy) : schema.conforms p.toResource = true
```

AiAgentTerraform.Tf.Aws.Data.IamPolicyDocument.toResource_conforms

Policy documents conform to their schema.

```
theorem toResource_conforms (d : IamPolicyDocument) : schema.conforms d.toResource = true
```

AiAgentTerraform.Tf.Aws.Data.Route53Zone.toResource_conforms

Hosted-zone lookups conform to their schema.

```
theorem toResource_conforms (z : Route53Zone) : schema.conforms z.toResource = true
```

AiAgentTerraform.Tf.Aws.Data.Vpc.toResource_conforms

VPC lookups conform to their schema.

```
theorem toResource_conforms (v : Vpc) : schema.conforms v.toResource = true
```

A.6 AiAgentTerraform/Model/Aws/Iam.lean

AiAgentTerraform.Tf.Aws.IamRole.toResource_conforms

Every IAM role model elaborates to a resource that conforms to the `aws_iam_role` schema.

```
theorem toResource_conforms (r : IamRole) :
```

```
  schema.conforms r.toResource = true
```

AiAgentTerraform.Tf.Aws.IamPolicy.toResource_conforms

Policies conform to their schema.

```
theorem toResource_conforms (p : IamPolicy) :
```

```
  schema.conforms p.toResource = true
```

AiAgentTerraform.Tf.Aws.IamRolePolicy.toResource_conforms

Inline role policies conform to their schema.

```
theorem toResource_conforms (p : IamRolePolicy) :
```

```
  schema.conforms p.toResource = true
```

AiAgentTerraform.Tf.Aws.IamRolePolicyAttachment.toResource_conforms

Attachments conform to their schema.

```
theorem toResource_conforms (a : IamRolePolicyAttachment) :
```

```
  schema.conforms a.toResource = true
```

AiAgentTerraform.Tf.Aws.IamInstanceProfile.toResource_conforms

Instance profiles conform to their schema.

```
theorem toResource_conforms (p : IamInstanceProfile) :
```

```
  schema.conforms p.toResource = true
```

AiAgentTerraform.Tf.Aws.IamUser.toResource_conforms

Users conform to their schema.

```
theorem toResource_conforms (u : IamUser) :
```

```
  schema.conforms u.toResource = true
```

A.7 AiAgentTerraform/Model/Aws/Modules.lean

AiAgentTerraform.Tf.Aws.Modules.Component.source_toModuleCall

A component's source path is always relative to the stack that calls it.

```
theorem Component.source_toModuleCall (c : Component) :
```

```
  c.toModuleCall.source = "./components/" ++ c.dir
```

A.8 AiAgentTerraform/Model/Aws/Network.lean

AiAgentTerraform.Tf.Aws.LbTargetGroup.toResource_conforms

Target groups conform to their schema.

```
theorem toResource_conforms (g : LbTargetGroup) : schema.conforms g.toResource = true
```

AiAgentTerraform.Tf.Aws.LbListener.toResource_conforms

Listeners conform to their schema.

```
theorem toResource_conforms (l : LbListener) : schema.conforms l.toResource = true
```

AiAgentTerraform.Tf.Aws.LbListenerRule.toResource_conforms

Listener rules conform to their schema.

```
theorem toResource_conforms (r : LbListenerRule) : schema.conforms r.toResource = true
```

AiAgentTerraform.Tf.Aws.Route53Zone.toResource_conforms

Zones conform to their schema.

```
theorem toResource_conforms (z : Route53Zone) : schema.conforms z.toResource = true
```

AiAgentTerraform.Tf.Aws.Route53Record.toResource_conforms

Records conform to their schema.

```
theorem toResource_conforms (r : Route53Record) : schema.conforms r.toResource = true
```

AiAgentTerraform.Tf.Aws.CloudflareRecord.toResource_conforms

Cloudflare records conform to their schema.

```
theorem toResource_conforms (r : CloudflareRecord) : schema.conforms r.toResource = true
```

A.9 AiAgentTerraform/Model/Aws/Observability.lean

AiAgentTerraform.Tf.Aws.CloudwatchLogGroup.toResource_conforms

Log groups conform to their schema.

```
theorem toResource_conforms (g : CloudwatchLogGroup) : schema.conforms g.toResource = true
```

AiAgentTerraform.Tf.Aws.CloudwatchMetricAlarm.toResource_conforms

Alarms conform to their schema.

```
theorem toResource_conforms (a : CloudwatchMetricAlarm) : schema.conforms a.toResource = true
```

AiAgentTerraform.Tf.Aws.CloudwatchDashboard.toResource_conforms

Dashboards conform to their schema.

```
theorem toResource_conforms (d : CloudwatchDashboard) : schema.conforms d.toResource = true
```

AiAgentTerraform.Tf.Aws.CloudwatchEventRule.toResource_conforms

Event rules conform to their schema.

```
theorem toResource_conforms (r : CloudwatchEventRule) : schema.conforms r.toResource = true
```

AiAgentTerraform.Tf.Aws.CloudwatchEventTarget.toResource_conforms

Event targets conform to their schema.

```
theorem toResource_conforms (t : CloudwatchEventTarget) : schema.conforms t.toResource = true
```

AiAgentTerraform.Tf.Aws.SnsTopic.toResource_conforms

Topics conform to their schema.

```
theorem toResource_conforms (t : SnsTopic) : schema.conforms t.toResource = true
```

AiAgentTerraform.Tf.Aws.XraySamplingRule.toResource_conforms

Sampling rules conform to their schema.

```
theorem toResource_conforms (r : XraySamplingRule) : schema.conforms r.toResource = true
```

A.10 AiAgentTerraform/Model/Aws/Serverless.lean

AiAgentTerraform.Tf.Aws.LambdaFunction.toResource_conforms

Lambda functions conform to their schema.

```
theorem toResource_conforms (f : LambdaFunction) : schema.conforms f.toResource = true
```

AiAgentTerraform.Tf.Aws.CognitoUserPool.toResource_conforms

User pools conform to their schema.

```
theorem toResource_conforms (p : CognitoUserPool) : schema.conforms p.toResource = true
```

AiAgentTerraform.Tf.Aws.GithubActionsSecret.toResource_conforms

GitHub secrets conform to their schema.

```
theorem toResource_conforms (s : GithubActionsSecret) : schema.conforms s.toResource = true
```

A.11 AiAgentTerraform/Model/Aws/Ssm.lean

AiAgentTerraform.Tf.Aws.SsmDocument.toResource_conforms

Documents conform to their schema.

```
theorem toResource_conforms (d : SsmDocument) : schema.conforms d.toResource = true
```

AiAgentTerraform.Tf.Aws.SsmParameter.toResource_conforms

Parameters conform to their schema.

```
theorem toResource_conforms (p : SsmParameter) : schema.conforms p.toResource = true
```

AiAgentTerraform.Tf.Aws.SsmAssociation.toResource_conforms

Associations conform to their schema.

```
theorem toResource_conforms (a : SsmAssociation) : schema.conforms a.toResource = true
```

AiAgentTerraform.Tf.Aws.SsmPatchBaseline.toResource_conforms

Baselines conform to their schema.

```
theorem toResource_conforms (b : SsmPatchBaseline) : schema.conforms b.toResource = true
```

AiAgentTerraform.Tf.Aws.SsmPatchGroup.toResource_conforms

Patch groups conform to their schema.

```
theorem toResource_conforms (g : SsmPatchGroup) : schema.conforms g.toResource = true
```

AiAgentTerraform.Tf.Aws.SsmMaintenanceWindow.toResource_conforms

Windows conform to their schema.

```
theorem toResource_conforms (w : SsmMaintenanceWindow) : schema.conforms w.toResource = true
```

AiAgentTerraform.Tf.Aws.SsmMaintenanceWindowTarget.toResource_conforms

A maintenance-window target conforms to its schema as soon as it selects at least one set of instances, which the provider requires.

```
theorem toResource_conforms (t : SsmMaintenanceWindowTarget) (h : t.targets ≠ []) :  
  schema.conforms t.toResource = true
```

AiAgentTerraform.Tf.Aws.SsmMaintenanceWindowTask.toResource_conforms

Tasks conform to their schema.

```
theorem toResource_conforms (t : SsmMaintenanceWindowTask) :  
  schema.conforms t.toResource = true
```

A.12 AiAgentTerraform/Model/Aws/Storage.lean

AiAgentTerraform.Tf.Aws.S3Bucket.toResource_conforms

Buckets conform to their schema.

```
theorem toResource_conforms (b : S3Bucket) : schema.conforms b.toResource = true
```

AiAgentTerraform.Tf.Aws.S3BucketVersioning.toResource_conforms

Versioning configurations conform to their schema.

```
theorem toResource_conforms (v : S3BucketVersioning) : schema.conforms v.toResource = true
```


AiAgentTerraform.Tf.Aws.S3BucketPolicy.toResource_conforms

Bucket policies conform to their schema.

```
theorem toResource_conforms (p : S3BucketPolicy) : schema.conforms p.toResource = true
```

AiAgentTerraform.Tf.Aws.S3BucketLifecycleConfiguration.toResource_conforms

Lifecycle configurations conform to their schema.

```
theorem toResource_conforms (c : S3BucketLifecycleConfiguration) :  
  schema.conforms c.toResource = true
```

AiAgentTerraform.Tf.Aws.KmsKey.toResource_conforms

Keys conform to their schema.

```
theorem toResource_conforms (k : KmsKey) : schema.conforms k.toResource = true
```

AiAgentTerraform.Tf.Aws.SecretsmanagerSecret.toResource_conforms

Secrets conform to their schema.

```
theorem toResource_conforms (s : SecretsmanagerSecret) : schema.conforms s.toResource = true
```

A.13 AiAgentTerraform/Model/Introspector.lean

AiAgentTerraform.Tf.Introspector.Quine.toResource_conforms

The quine conforms to its schema.

```
theorem toResource_conforms (q : Quine) : schema.conforms q.toResource = true
```

A.14 AiAgentTerraform/Model/Packer.lean

AiAgentTerraform.Packer.images_wellFormed

Both templates are well formed: every build builds a source the template defines, and every source is built.

```
theorem images_wellFormed :  
  swarmsImage.wellFormed = true ∧ fastapiImage.wellFormed = true
```

AiAgentTerraform.Packer.swarmsImage_runs_install_script

The image the deployment boots runs the very installer that the systemd bootstrap runs at first boot, so the two agree on where the API lives.

```
theorem swarmsImage_runs_install_script :  
  "sudo bash -x /opt/swarms/api/install.sh" ∈ swarmsImage.commands
```

AiAgentTerraform.Packer.swarmsImage_root_volume

The image is built on a 30 GiB gp3 root volume that is deleted with the builder instance.

```
theorem swarmsImage_root_volume :  
  swarmsImage.sources.flatMap AmazonEbs.blockDevices =  
    [{ deviceName := "/dev/sda1", volumeSize := 30, volumeType := "gp3"  
      , deleteOnTermination := true }]
```

AiAgentTerraform.Packer.imageFiles_paths

Every image file has a non-empty path and is HCL.

```
theorem imageFiles_paths : ∀ f ∈ imageFiles, f.path ≠ ""
```

A.15 AiAgentTerraform/Model/Properties.lean

AiAgentTerraform.Tf.Aws.registry_covers_upstream_resources

Every managed resource type that occurs in the upstream repository has a schema in the registry.

```
theorem registry_covers_upstream_resources :  
  upstreamResourceTypes.all (fun t => managedTypes.contains t) = true
```

AiAgentTerraform.Tf.Aws.registry_covers_upstream_data

Every data-source type that occurs upstream has a schema too.

```
theorem registry_covers_upstream_data :  
  upstreamDataTypes.all (fun t => dataTypes.contains t) = true
```

AiAgentTerraform.Tf.Aws.registry_covers_prose_resources

The self-referential resource of the root main.tf is modelled too.

```
theorem registry_covers_prose_resources :  
  Introspector.proseResourceTypes.all (fun t => managedTypes.contains t) = true
```

AiAgentTerraform.Tf.Aws.registry_size

The registry models forty managed resource types and thirteen data-source types.

```
theorem registry_size : managedTypes.length = 40 ∧ dataTypes.length = 13
```

AiAgentTerraform.Tf.Aws.schemaFor_typeName

No resource type is modelled twice: looking a type up in the registry finds the schema of that very type.

```
theorem schemaFor_typeName (isData : Bool) (t : String) :  
  ∀ s ∈ schemaFor? isData t, s.typeName = t ∧ s.dataSource = isData
```

AiAgentTerraform.Tf.Swarm.swarmSystem_problems

Nothing is wrong with the modelled system: no duplicate addresses, no dangling references, and every resource conforms to the schema of its type.

```
theorem swarmSystem_problems : swarmSystem.problems registry = []
```

AiAgentTerraform.Tf.Swarm.swarmSystem_valid

Consequently the system is valid.

```
theorem swarmSystem_valid : swarmSystem.valid registry = true
```

AiAgentTerraform.Tf.Swarm.swarmSystem_all_types_modelled

Every resource type used by the modelled system is one the registry models: nothing is left unchecked.

```
theorem swarmSystem_all_types_modelled :  
  swarmSystem.parts.all (fun p => (p.2.unmodelledTypes registry).isEmpty) = true
```

AiAgentTerraform.Tf.Swarm.swarmSystem_parts

The system consists of the root stack and six components.

```
theorem swarmSystem_parts : swarmSystem.parts.length = 7
```

AiAgentTerraform.Tf.Swarm.swarmSystem_objectCount

It models forty objects: resources, data sources and module calls.

```
theorem swarmSystem_objectCount : swarmSystem.objectCount = 40
```

AiAgentTerraform.Tf.Swarm.swarmSystem_paths_nonempty

Every file the system generates has a non-empty path.

```
theorem swarmSystem_paths_nonempty : ∀ f ∈ swarmSystem.files, f.path ≠ ""
```

AiAgentTerraform.Tf.Swarm.stacks_no_duplicate_addresses

Each stack declares each address only once.

```
theorem stacks_no_duplicate_addresses :  
  swarmSystem.parts.all (fun p => p.2.duplicateAddresses.isEmpty) = true
```

AiAgentTerraform.Tf.Swarm.stacks_no_dangling_refs

Every reference in every stack resolves to something the stack declares.

```
theorem stacks_no_dangling_refs :  
  swarmSystem.parts.all (fun p => p.2.danglingRefs.isEmpty) = true
```

AiAgentTerraform.Tf.Swarm.listener_forwards_to_target_group

The HTTPS listener forwards to the API target group.

```
theorem listener_forwards_to_target_group :  
  apiListener.defaultAction.forwardsTo = some apiTargetGroup.arn
```

AiAgentTerraform.Tf.Swarm.listener_uses_api_certificate

The listener terminates TLS with the certificate issued for the API domain.

theorem listener_uses_api_certificate :

```
  apiListener.certificateArn = some apiCert.certificateArn
```

AiAgentTerraform.Tf.Swarm.autoscaling_attached_to_alb

The auto scaling group registers its instances with the target group that the load balancer's listener forwards to.

theorem autoscaling_attached_to_alb :

```
  apiAutoscaling.targetGroupArn = some (Expr.varRef .string "target_group_arn") ^
  (autoscalingCall.args.find? (fun a => a.1 == "target_group_arn")).isSome = true
```

AiAgentTerraform.Tf.Swarm.internal_group_only_from_external

The internal security group accepts traffic only from the external one, and that is the group the load balancer runs in.

theorem internal_group_only_from_external :

```
  internalGroup.ingressCidrBlocks = [] ^
  internalGroup.ingressFromGroups.map (fun i => i.sourceSecurityGroupId.render) =
  [externalGroup.id.render]
```

AiAgentTerraform.Tf.Swarm.instances_in_internal_group

The API instances are launched into the internal security group, not the public one.

theorem instances_in_internal_group :

```
  (launchTemplateCall.args.find? (fun a => a.1 == "security_group_id")).map
  (fun a => a.2.expr.render) =
  some (Expr.moduleOut .string "security" "internal_security_group_id" |> Expr.render)
```

AiAgentTerraform.Tf.Swarm.launch_template_user_data

The launch template boots the instances with the Lean-generated bootstrap script, base64-encoded as `user_data` requires.

theorem launch_template_user_data :

```
  apiLaunchTemplate.userData =
  some (Expr.call0f .string "base64encode" [.any (Expr.localRef .string "user_data")]) ^
  launchTemplateStack.locals.map LocalDef.name = ["user_data"]
```

AiAgentTerraform.Tf.Swarm.bootstrap_installs_api_unit

That bootstrap is exactly the one the systemd deployment defines: it installs and starts the API unit.

theorem bootstrap_installs_api_unit :

```
  Shell.Stmt.installUnit Deployment.swarmsApiUnit ∈
  (Deployment.bootstrapScript "master" "/opt/swarms/api/install.sh"
  "/cloudwatch-agent/config").stmts
```

AiAgentTerraform.Tf.Swarm.cloudwatch_config_parameter_matches

The CloudWatch agent configuration the bootstrap fetches from the parameter store is the parameter the operations component creates.

theorem cloudwatch_config_parameter_matches :

```
  cloudwatchConfig.parameterName = Expr.str "/cloudwatch-agent/config"
```

AiAgentTerraform.Tf.Swarm.instance_role_policies

The instance role can be assumed by EC2 and carries the two managed policies the deployment needs.

theorem instance_role_policies :

```
  instanceRole.assumeRolePolicy.render = (assumeRoleForService "ec2.amazonaws.com").render ^
  ssmCoreAttachment.policyArn.render =
  "\{"arn:aws:iam::aws:policy/AmazonSSMManagedInstanceCore\}" ^
  cloudwatchAttachment.policyArn.render =
  "\{"arn:aws:iam::aws:policy/CloudWatchAgentServerPolicy\}"
```

AiAgentTerraform.Tf.Swarm.api_record_points_at_alb

The DNS name of the API is a CNAME onto the load balancer.

```
theorem api_record_points_at_alb :
  apiRecord.recordType = Expr.str "CNAME" ∧
  apiRecord.records.map (fun r => r.render) = some (Expr.listOf .string [alb.dnsName]).render
```

AiAgentTerraform.Tf.Swarm.root_calls_components

The root stack calls every component, and each component call is a directory of this repository.

```
theorem root_calls_components :
  rootStack.modules.map ModuleCall.source =
  [ "./components/security", "./components/keypairs", "./components/launch_template"
  , "./components/application_load_balancer", "./components/autoscaling_group"
  , "./components/operations" ]
```

AiAgentTerraform.Tf.Swarm.components_are_generated

Every component the root stack calls is generated as a file of the system.

```
theorem components_are_generated :
  components.all (fun c =>
    swarmSystem.parts.any (fun p => p.1 == componentPath c.dir)) = true
```

AiAgentTerraform.Tf.Account.accountSystem_problems

The account layer and the machine image are well formed too.

```
theorem accountSystem_problems : accountSystem.problems registry = []
```

AiAgentTerraform.Tf.Account.accountStack_reads_only

The account stack creates nothing: it only reads the account.

```
theorem accountStack_reads_only :
  accountStack.resources = [] ∧ accountStack.modules = [] ∧
  accountStack.dataSources.length = 6
```

AiAgentTerraform.Tf.Account.accountStack_dataSources_are_data

Every data source of the account stack is a lookup, i.e. its address is in the data. namespace.

```
theorem accountStack_dataSources_are_data :
  accountStack.dataSources.all (fun r => r.address.scope == .data) = true
```

AiAgentTerraform.Tf.Account.testInstance_uses_ubuntu_image

The test instance boots from the image the stack looks up.

```
theorem testInstance_uses_ubuntu_image :
  testInstance.ami.render = ubuntuImage.id.render
```

AiAgentTerraform.Tf.Account.ubuntuImage_selection

The image is the most recent Canonical image matching the Focal server name pattern.

```
theorem ubuntuImage_selection :
  ubuntuImage.owners.map (fun e => e.render) = some (Expr.strs ["099720109477"]).render ∧
  ubuntuImage.filters.map (fun f => f.filterName.render) = [(Expr.str "name").render]
```

AiAgentTerraform.Tf.Introspector.quineStack_problems

The self-referential resource is well formed against its own schema.

```
theorem quineStack_problems : quineStack.problems registry = []
```

AiAgentTerraform.Tf.Introspector.systemG_bounds

The quine names itself G, and bounds its proof by the two variables the stack declares.

```
theorem systemG_bounds :
  systemG.systemName.render = (Expr.str "G").render ∧
  systemG.probability.map (fun e => e.render) = some (Expr.varRef .number varP.name).render ∧
  systemG.threshold.map (fun e => e.render) = some (Expr.varRef .number varM.name).render
```

AiAgentTerraform.Tf.Introspector.systemG_self_modifying

It is self-modifying, and says so.

```
theorem systemG_self_modifying :
  systemG.selfModifying.map (fun m => m.enabled.render) = some (Expr.bool true).render
```

AiAgentTerraform.Tf.Deploy.deploySystem_problems

The roles component and the second environment are well formed.

```
theorem deploySystem_problems : deploySystem.problems registry = []
```

AiAgentTerraform.Tf.Deploy.role_carries_both_policies

The customer-managed policy is exactly the document the component builds, and the role carries both it and the AWS-managed SSM policy.

```
theorem role_carries_both_policies :  
  ssmPolicy.policy.render = ssmPolicyDocument.json.render ^  
  ssmPolicyAttachment.policyArn.render = ssmPolicy.arn.render ^  
  ssmCoreAttachment.policyArn.render = managedSsmCore.arn.render
```

AiAgentTerraform.Tf.Deploy.profile_and_attachments_use_the_role

Both attachments and the instance profile refer to the one role the component creates.

```
theorem profile_and_attachments_use_the_role :  
  [ssmPolicyAttachment.role.render, ssmCoreAttachment.role.render, ssmProfile.role.render].all  
    (fun r => r == ssmRole.nameRef.render) = true
```

AiAgentTerraform.Tf.Deploy.deploy_for_each_same_set

The deploy root creates one launch template and one auto scaling group per instance type, over the same set.

```
theorem deploy_for_each_same_set :  
  launchTemplateCall.forEach.map (fun e => e.render) = some instanceTypeSet.render ^  
  autoscalingCall.forEach.map (fun e => e.render) = some instanceTypeSet.render
```

AiAgentTerraform.Tf.Deploy.asg_uses_matching_launch_template

Each auto scaling group runs on the launch template of its own instance type.

```
theorem asg_uses_matching_launch_template :  
  (autoscalingCall.args.find? (fun a => a.1 == "launch_template_id")).map  
    (fun a => a.2.expr.render) =  
    some "module.lt_dynamic[each.key].launch_template_id"
```

AiAgentTerraform.Tf.Deploy.deploy_reads_vpc

The deploy root reads the VPC instead of creating one.

```
theorem deploy_reads_vpc :  
  deployStack.dataSources.map Resource.type = ["aws_vpc"] ^  
  deployStack.resources = []
```

AiAgentTerraform.Tf.fullSystem_problems

Nothing is wrong with any part of the modelled system.

```
theorem fullSystem_problems : fullSystem.problems registry = []
```

AiAgentTerraform.Tf.fullSystem_valid

Consequently the whole model is valid.

```
theorem fullSystem_valid : fullSystem.valid registry = true
```

AiAgentTerraform.Tf.fullSystem_all_types_modelled

No stack of the model uses a resource type the registry does not model.

```
theorem fullSystem_all_types_modelled :  
  fullSystem.parts.all (fun p => (p.2.unmodelledTypes registry).isEmpty) = true
```

AiAgentTerraform.Tf.fullSystem_parts

The model consists of twelve stacks.

```
theorem fullSystem_parts : fullSystem.parts.length = 12
```

AiAgentTerraform.Tf.fullSystem_objectCount

It models sixty objects: resources, data sources and module calls.

```
theorem fullSystem_objectCount : fullSystem.objectCount = 60
```

AiAgentTerraform.Tf.fullSystem_stacks_well_scoped

Every stack of the model declares each address only once, and every reference resolves to something the stack declares.

```
theorem fullSystem_stacks_well_scoped :
  fullSystem.parts.all (fun p => p.2.duplicateAddresses.isEmpty) = true ∧
  fullSystem.parts.all (fun p => p.2.danglingRefs.isEmpty) = true
```

AiAgentTerraform.Tf.fullSystem_paths_distinct

Every file the model generates has a distinct, non-empty path.

```
theorem fullSystem_paths_distinct :
  (∀ f ∈ fullSystem.files, f.path ≠ "") ∧
  (fullSystem.files.map GenFile.path).eraseDups.length = fullSystem.files.length
```

A.16 AiAgentTerraform/Model/Resource.lean

AiAgentTerraform.Tf.all_map_true

If a function always produces values satisfying p , mapping it over any list gives a list all of whose elements satisfy p .

```
theorem all_map_true {α β : Type} {f : α → β} {p : β → Bool} (h : ∀ x, p (f x) = true) :
  ∀ xs : List α, (xs.map f).all p = true
```

AiAgentTerraform.Tf.all_optAttr

An optional attribute is acceptable as soon as it is acceptable when set.

```
theorem all_optAttr {t : Ty} {n : String} {p : String × AnyE → Bool} {o : Option (Expr t)}
  (h : ∀ e : Expr t, p (n, AnyE.of e) = true) : (optAttr n o).all p = true
```

AiAgentTerraform.Tf.all_option_toList

An optional block is acceptable as soon as it is acceptable when present.

```
theorem all_option_toList {α β : Type} {f : α → β} {p : β → Bool} {o : Option α}
  (h : ∀ x, p (f x) = true) : ((o.map f).toList).all p = true
```

AiAgentTerraform.Tf.filter_map_eq_nil

A filtered-out list maps to the empty list.

```
theorem filter_map_eq_nil {α β : Type} {p : α → Bool} {f : α → β} :
  ∀ {xs : List α}, xs.all (fun x => !p x) = true → (xs.filter p).map f = []
```

AiAgentTerraform.Tf.ResourceSchema.errors_eq_nil_of_conforms

A conforming resource has no diagnostics: `conforms` is the Boolean form of "errors is empty".

```
theorem errors_eq_nil_of_conforms {s : ResourceSchema} {r : Resource}
  (h : s.conforms r = true) : s.errors r = []
```

A.17 AiAgentTerraform/Zkp/Bridge.lean

AiAgentTerraform.Zkp.eq_nil_of_eraseDups_nil

```
private theorem eq_nil_of_eraseDups_nil {α : Type} [BEq α] {l : List α}
  (h : l.eraseDups = []) : l = []
```

AiAgentTerraform.Zkp.conforms_of_errors_nil

A resource with no schema diagnostics conforms to the schema: the converse of `ResourceSchema.errors_eq_nil_of_conforms`.

```
theorem conforms_of_errors_nil {s : ResourceSchema} {r : Resource} (h : s.errors r = []) :
  s.conforms r = true
```

AiAgentTerraform.Zkp.claimOfResource_conforms

If the registry models the type of a resource and the schema check finds no fault with it, the claim the roll-up carries is marked type-correct.

```
theorem claimOfResource_conforms {registry : List ResourceSchema} {r : Resource}
  {sc : ResourceSchema}
  (hfind : registry.find? (fun s =>
    s.typeName == r.type && s.dataSource == (r.address.scope == .data)) = some sc)
  (herr : sc.errors r = []) : (claimOfResource registry r).conforms = true
```

AiAgentTerraform.Zkp.stack_claims_conform

A stack the model accepts — no schema violations and no unmodelled type — has all of its objects' claims marked type-correct.

```
theorem stack_claims_conform {registry : List ResourceSchema} {s : Stack}
  (herr : s.schemaErrors registry = []) (hmod : s.unmodelledTypes registry = []) :
  ∀ r ∈ s.resources ++ s.dataSources, (claimOfResource registry r).conforms = true
```

AiAgentTerraform.Zkp.system_claims_conform

The whole modelled system agrees. Because the model proves that the system has no problems and no unmodelled type, every claim the roll-up makes about a modelled object is marked type-correct — the flag is the model's own schema check, not an independent assertion.

```
theorem system_claims_conform :
  ∀ p ∈ fullSystem.parts, ∀ r ∈ p.2.resources ++ p.2.dataSources,
    (claimOfResource Aws.registry r).conforms = true
```

A.18 AiAgentTerraform/Zkp/Claim.lean

AiAgentTerraform.Zkp.Usage.add_def

```
@[simp] theorem add_def (a b : Usage) : a + b = a.add b
```

AiAgentTerraform.Zkp.Usage.zero_def

```
@[simp] theorem zero_def : (0 : Usage) = Usage.zero
```

AiAgentTerraform.Zkp.Usage.sum_nil

```
@[simp] theorem sum_nil : sum [] = 0
```

AiAgentTerraform.Zkp.Usage.sum_cons

```
@[simp] theorem sum_cons (u : Usage) (l : List Usage) : sum (u :: l) = u + sum l
```

AiAgentTerraform.Zkp.Usage.zero_add'

```
theorem zero_add' (u : Usage) : Usage.zero.add u = u
```

AiAgentTerraform.Zkp.Usage.add_assoc'

```
theorem add_assoc' (a b c : Usage) : (a.add b).add c = a.add (b.add c)
```

AiAgentTerraform.Zkp.Usage.sum_append

```
theorem sum_append (l1 l2 : List Usage) : sum (l1 ++ l2) = sum l1 + sum l2
```

AiAgentTerraform.Zkp.Rollup.leaves_leaf

```
@[simp] theorem leaves_leaf (a : Attested) : (leaf a).leaves = [a]
```

AiAgentTerraform.Zkp.Rollup.leavesList_eq

```
theorem leavesList_eq (ks : List Rollup) : leavesList ks = ks.flatMap leaves
```

AiAgentTerraform.Zkp.Rollup.leaves_node

```
@[simp] theorem leaves_node (l : String) (ks : List Rollup) :
  (node l ks).leaves = ks.flatMap leaves
```

AiAgentTerraform.Zkp.Rollup.flatMap_singleton_eq_map

Flattening singletons is mapping.

```
@[simp] theorem flatMap_singleton_eq_map {α β : Type*} (g : α → β) (l : List α) :
  l.flatMap (fun a => [g a]) = l.map g
```

AiAgentTerraform.Zkp.Rollup.flatMap_leaves_leaf

Flattening a list of single-claim roll-ups gives the claims themselves.

```
@[simp] theorem flatMap_leaves_leaf {α : Type*} (f : α → Attested) (l : List α) :
  (l.map (fun a => Rollup.leaf (f a))).flatMap leaves = l.map f
```

AiAgentTerraform.Zkp.Rollup.leaves_append_node

```
theorem leaves_append_node (l : String) (ks : List Rollup) (t : Rollup) :
  (node l (t :: ks)).leaves = t.leaves ++ (node l ks).leaves
```

AiAgentTerraform.Zkp.Rollup.total_node

theorem total_node (f : Claim → Nat) (l : String) (ks : List Rollup) :
total f (node l ks) = (ks.map (total f)).sum

AiAgentTerraform.Zkp.Rollup.costCents_node

theorem costCents_node (l : String) (ks : List Rollup) :
costCents (node l ks) = (ks.map costCents).sum

AiAgentTerraform.Zkp.Rollup.usage_node

theorem usage_node (l : String) (ks : List Rollup) :
usage (node l ks) = Usage.sum (ks.map usage)

AiAgentTerraform.Zkp.Rollup.allConform_node

theorem allConform_node (l : String) (ks : List Rollup) :
allConform (node l ks) = ks.all allConform

AiAgentTerraform.Zkp.Rollup.size_node

theorem size_node (l : String) (ks : List Rollup) :
size (node l ks) = (ks.map size).sum

AiAgentTerraform.Zkp.Rollup.commitTo_eq

The roll-up opens to the aggregate. The commitment of a tree is a single Pedersen commitment, to the total of the committed quantity, blinded by the total of the nonces. This is what makes an aggregate figure provable without revealing the individual ones.

theorem commitTo_eq (P : Params modulus M) (f : Claim → Fp) (t : Rollup) :
t.commitTo P f = P.commit ((t.claims.map f).sum) t.nonceSum

AiAgentTerraform.Zkp.Rollup.commitTo_node

Commitments roll up. The commitment of a node is the sum of the commitments of its children: each part of the system can publish its own aggregate, and the aggregates combine into the system's without reopening anything.

theorem commitTo_node (P : Params modulus M) (f : Claim → Fp) (l : String) (ks : List Rollup) :
commitTo P f (node l ks) = (ks.map (commitTo P f)).sum

AiAgentTerraform.Zkp.Rollup.sum_costFp_eq

Casting a total of costs into the field is the total of the cast costs.

theorem sum_costFp_eq (l : List Claim) :
((l.map Claim.costCents).sum : Fp) = (l.map Claim.costFp).sum

AiAgentTerraform.Zkp.Rollup.cost_commitment

The commitment to the cost of the system opens exactly to its total cost.

theorem cost_commitment (P : Params modulus M) (t : Rollup) :
t.commitTo P Claim.costFp = P.commit ((t.costCents : Fp)) t.nonceSum

AiAgentTerraform.Zkp.Rollup.cost_binding

Binding of the published total. If the published commitment of the system also opens to a cost different from the one the resources attested to, then the discrete logarithm of g in base h is exposed. So, under the discrete-logarithm assumption, the aggregate cost behind the published commitment is the sum of the per-resource costs.

theorem cost_binding (P : Params modulus M) (t : Rollup) {c' r' : Fp}
(heq : P.commit c' r' = t.commitTo P Claim.costFp) (hne : c' ≠ (t.costCents : Fp)) :
P.g = ((t.nonceSum - r') / (c' - (t.costCents : Fp))) • P.h

AiAgentTerraform.Zkp.Rollup.stepProof_verify

Each step's own proof is accepted against its own commitment.

theorem stepProof_verify (P : Params modulus M) (f : Claim → Fp) (a : Attested) (r1 r2 e : Fp) :
P.verify (P.commit (f a.claim) a.nonce) (stepProof P f a r1 r2 e)

AiAgentTerraform.Zkp.Rollup.step_proofs_aggregate

The sub-proofs roll up. If every step of the tree proves possession of the opening of its own commitment, answering a common challenge, then the combination of those proofs is accepted against the commitment of the whole tree. The aggregate argument is assembled from the per-step arguments; no step has to reveal its claim, and no aggregate has to be reopened.

```
theorem step_proofs_aggregate (P : Params modulus M) (f : Claim → Fp) (t : Rollup)
  (rand : Attested → Fp × Fp) (e : Fp) :
  P.verify (t.commitTo P f)
    (Transcript.combineList e
      (t.leaves.map (fun a => stepProof P f a (rand a).1 (rand a).2 e)))
```

AiAgentTerraform.Zkp.Rollup.prove_verify

The system can prove possession of its own roll-up. The proof produced from the claims of the tree is accepted against the tree's published commitment — and by `honest_verifier_zk` it discloses neither the individual claims nor the aggregate.

```
theorem prove_verify (P : Params modulus M) (chal : M → M → Fp) (f : Claim → Fp) (t : Rollup)
  (a1 a2 : Fp) :
  verify P chal (t.commitTo P f) (prove P chal f t a1 a2)
```

A.19 AiAgentTerraform/Zkp/Commit.lean

AiAgentTerraform.Zkp.Params.commit_zero

```
@[simp] theorem Params.commit_zero (P : Params q M) : P.commit 0 0 = 0
```

AiAgentTerraform.Zkp.Params.commit_add

Commitments are additively homomorphic.

```
theorem Params.commit_add (P : Params q M) (m1 r1 m2 r2 : ZMod q) :
  P.commit m1 r1 + P.commit m2 r2 = P.commit (m1 + m2) (r1 + r2)
```

AiAgentTerraform.Zkp.Params.sum_commit

The sum of a list of commitments is the commitment of the sums.

```
theorem Params.sum_commit (P : Params q M) :
  ∀ l : List (ZMod q × ZMod q),
    (l.map (fun p => P.commit p.1 p.2)).sum =
      P.commit (l.map Prod.fst).sum (l.map Prod.snd).sum
```

AiAgentTerraform.Zkp.Params.sum_commit_map

The same, for a list of arbitrary data with a value and a randomness attached to each element.

```
theorem Params.sum_commit_map {α : Type*} (P : Params q M) (v ρ : α → ZMod q) (l : List α) :
  (l.map (fun a => P.commit (v a) (ρ a))).sum =
    P.commit ((l.map v).sum) ((l.map ρ).sum)
```

AiAgentTerraform.Zkp.Params.commit_hiding

Perfect hiding. Given that $g = s \bullet h$, for any two values the bijection `hidingEquiv` maps the randomness of a commitment to m_1 onto randomness giving the very same commitment to m_2 . A commitment is therefore distributed identically whatever the committed value is.

```
theorem Params.commit_hiding (P : Params q M) {s : ZMod q} (hs : P.g = s • P.h)
  (m1 m2 r : ZMod q) :
  P.commit m1 r = P.commit m2 (Params.hidingEquiv (q := q) m1 m2 s r)
```

AiAgentTerraform.Zkp.Params.hidingEquiv_bijective

The hiding map is a bijection of the randomness space.

```
theorem Params.hidingEquiv_bijective (m1 m2 s : ZMod q) :
  Function.Bijective (Params.hidingEquiv (q := q) m1 m2 s)
```

AiAgentTerraform.Zkp.Params.commit_binding

Binding. Two openings of one commitment with different values give the discrete logarithm of g in base h ; so an adversary that breaks binding solves the discrete-logarithm problem in M .

```
theorem Params.commit_binding (P : Params q M) {m1 r1 m2 r2 : ZMod q}
  (heq : P.commit m1 r1 = P.commit m2 r2) (hm : m1 ≠ m2) :
  P.g = ((r2 - r1) / (m1 - m2)) • P.h
```

AiAgentTerraform.Zkp.Params.value_unique

Contrapositive of binding in the form the roll-up uses: if no discrete logarithm of g in base h is known to exist, a commitment determines the value it was made to.

```
theorem Params.value_unique (P : Params q M) (hdl :  $\forall s : \text{ZMod } q, P.g \neq s \bullet P.h$ )  
  {m1 r1 m2 r2 : ZMod q} (heq : P.commit m1 r1 = P.commit m2 r2) : m1 = m2
```

A.20 AiAgentTerraform/Zkp/Field.lean

AiAgentTerraform.Zkp.modulus_prime

```
theorem modulus_prime : Nat.Prime modulus
```

AiAgentTerraform.Zkp.digestFp_nil

```
theorem digestFp_nil : digestFp [] = 1
```

AiAgentTerraform.Zkp.digestFp_cons

```
theorem digestFp_cons (x : Fp) (xs : List Fp) :  
  digestFp (x :: xs) = xs.foldl digestStep (digestStep 1 x)
```

AiAgentTerraform.Zkp.digestFp_append

```
theorem digestFp_append (xs ys : List Fp) :  
  digestFp (xs ++ ys) = ys.foldl digestStep (digestFp xs)
```

AiAgentTerraform.Zkp.digestStep_injective

The step function is injective in the datum being folded in: two different field elements folded into the same accumulator give different digests.

```
theorem digestStep_injective (acc : Fp) : Function.Injective (digestStep acc)
```

AiAgentTerraform.Zkp.digestStep_injective_acc

The step function is injective in the accumulator: different histories with the same next datum stay different.

```
theorem digestStep_injective_acc (x : Fp) : Function.Injective (fun a => digestStep a x)
```

A.21 AiAgentTerraform/Zkp/Merkle.lean

AiAgentTerraform.Zkp.exists_path

Completeness of the opening. Every leaf of every tree has an authentication path that recomputes the root: any step of the system can be shown to belong to the published root, revealing only the digests of the siblings on its path.

```
theorem exists_path (H : Hash) :  
   $\forall (t : \text{MTree}), \forall v \in t.\text{leaves}, \exists p : \text{MPath}, \text{rootWith } H \ v \ p = t.\text{root } H$ 
```

AiAgentTerraform.Zkp.mem_leaves_of_rootWith

Soundness of the opening. If the compression function is injective, an authentication path of the right length that recomputes the root of a perfect tree can only have started from a value that really is a leaf of that tree.

```
theorem mem_leaves_of_rootWith (H : Hash)  
  (hij :  $\forall a \ b \ c \ d : \text{Fp}, H.h \ a \ b = H.h \ c \ d \rightarrow a = c \wedge b = d$ ) :  
   $\forall (n : \text{Nat}) (t : \text{MTree}), t.\text{Depth } n \rightarrow \forall (v : \text{Fp}) (p : \text{MPath}), p.\text{length} = n \rightarrow$   
     $\text{rootWith } H \ v \ p = t.\text{root } H \rightarrow v \in t.\text{leaves}$ 
```

AiAgentTerraform.Zkp.perfect_depth

```
theorem perfect_depth :  $\forall (d : \text{Nat}) (f : \text{Nat} \rightarrow \text{Fp}), (\text{perfect } d \ f).\text{Depth } d$ 
```

AiAgentTerraform.Zkp.mem_leaves_perfect

Every index below 2^d really is a leaf of the perfect tree.

```
theorem mem_leaves_perfect :  
   $\forall (d : \text{Nat}) (f : \text{Nat} \rightarrow \text{Fp}) (i : \text{Nat}), i < 2^d \rightarrow f \ i \in (\text{perfect } d \ f).\text{leaves}$ 
```

AiAgentTerraform.Zkp.exists_index_of_mem_leaves_perfect

Every leaf of a perfect tree is a value of the indexing family.

theorem exists_index_of_mem_leaves_perfect :

$\forall (d : \text{Nat}) (f : \text{Nat} \rightarrow \text{Fp}) (v : \text{Fp}), v \in (\text{perfect } d \text{ } f). \text{leaves} \rightarrow \exists i, f \ i = v$

AiAgentTerraform.Zkp.length_pathIn

@[simp] theorem length_pathIn (H : Hash) :

$\forall (d : \text{Nat}) (f : \text{Nat} \rightarrow \text{Fp}) (i : \text{Nat}), (\text{pathIn } H \ d \ f \ i). \text{length} = d$

AiAgentTerraform.Zkp.rootWith_pathIn

The computed path opens the leaf. Recomputing from the i -th leaf and `pathIn` gives the root of the tree.

theorem rootWith_pathIn (H : Hash) :

$\forall (d : \text{Nat}) (f : \text{Nat} \rightarrow \text{Fp}) (i : \text{Nat}), i < 2^d \rightarrow$

$\text{rootWith } H \ (f \ i) \ (\text{pathIn } H \ d \ f \ i) = (\text{perfect } d \ f). \text{root } H$

A.22 AiAgentTerraform/Zkp/Open.lean

AiAgentTerraform.Zkp.Rollup.tree_depth

theorem tree_depth (t : Rollup) (d : Nat) : (t.tree d).Depth d

AiAgentTerraform.Zkp.Rollup.length_stepDigests

theorem length_stepDigests (t : Rollup) : t.stepDigests.length = t.size

AiAgentTerraform.Zkp.rollup_step_opens

Every step of a roll-up can be opened. For each attested step there is an authentication path that recomputes the published root from that step's digest alone: the step is shown to be part of the system without revealing any other step, only the digests of the siblings on its path.

theorem rollup_step_opens {t : Rollup} {d : Nat} (hd : t.size $\leq 2^d$) :

$\forall a \in t.\text{leaves}, \exists p : \text{MPath},$

$\text{rootWith } \text{digestHash} \ (\text{stepDigest } a) \ p = t.\text{merkleRoot } d$

AiAgentTerraform.Zkp.rollup_open_sound

An accepted opening is an opening of a real step. If the compression function is injective, an authentication path of the tree's depth that recomputes the published root must have started from the digest of a step the roll-up really contains — or from the padding.

theorem rollup_open_sound (hinj : $\forall a \ b \ c \ e : \text{Fp}, \text{digestHash}.h \ a \ b = \text{digestHash}.h \ c \ e \rightarrow$

$a = c \wedge b = e$)

$\{t : \text{Rollup}\} \{d : \text{Nat}\} \{v : \text{Fp}\} \{p : \text{MPath}\} (hp : p.\text{length} = d)$

$(h : \text{rootWith } \text{digestHash} \ v \ p = t.\text{merkleRoot } d) :$

$v \in t.\text{stepDigests} \vee v = 0$

AiAgentTerraform.Zkp.systemRollup_fits

theorem systemRollup_fits (seed : Fp) : (systemRollup seed).size $\leq 2^{\text{systemDepth}}$

AiAgentTerraform.Zkp.system_step_opens

Every step of the system opens against the published root. Each of the 84 attested steps — modelled object, generated file or operation — has an authentication path recomputing `systemMerkleRoot`, revealing nothing about the other steps beyond the digests of the siblings on its path.

theorem system_step_opens (seed : Fp) :

$\forall a \in (\text{systemRollup } \text{seed}). \text{leaves}, \exists p : \text{MPath},$

$\text{rootWith } \text{digestHash} \ (\text{stepDigest } a) \ p = \text{systemMerkleRoot } \text{seed}$

AiAgentTerraform.Zkp.system_open_sound

No step can be invented after the fact. Under injectivity of the compression function, a depth-7 path accepted against the system's root opened one of the system's own step digests (or the padding).

theorem system_open_sound (hinj : $\forall a \ b \ c \ e : \text{Fp}, \text{digestHash}.h \ a \ b = \text{digestHash}.h \ c \ e \rightarrow$

$a = c \wedge b = e$)

$(\text{seed} : \text{Fp}) \{v : \text{Fp}\} \{p : \text{MPath}\} (hp : p.\text{length} = \text{systemDepth})$

$(h : \text{rootWith } \text{digestHash} \ v \ p = \text{systemMerkleRoot } \text{seed}) :$

$v \in (\text{systemRollup seed}).\text{stepDigests} \vee v = 0$

AiAgentTerraform.Zkp.Rollup.length_openStep

`@[simp] theorem Rollup.length_openStep (t : Rollup) (d i : Nat) :`
`(t.openStep d i).length = d`

AiAgentTerraform.Zkp.rollup_openStep_verifies

The computed opening is accepted. For every step index inside the tree, `openStep` recomputes the published root from that step's digest.

`theorem rollup_openStep_verifies {t : Rollup} {d i : Nat} (hi : i < 2 ^ d) :`
`rootWith digestHash (t.padDigest i) (t.openStep d i) = t.merkleRoot d`

AiAgentTerraform.Zkp.systemOpen_verifies

Each of the 84 steps has a printable opening that verifies against the system's root.

`theorem systemOpen_verifies (seed : Fp) {i : Nat} (hi : i < 2 ^ systemDepth) :`
`rootWith digestHash ((systemRollup seed).padDigest i) (systemOpen seed i) =`
`systemMerkleRoot seed`

AiAgentTerraform.Zkp.system_steps_signed

Every step of the system carries an operator signature. Signing each claim of the roll-up under one key produces signatures that all verify against the single public key, so the attestations are authenticated as well as committed.

`theorem system_steps_signed (K : SigKey modulus M) (chal : M → Fp → Fp) (seed : Fp)`
`(nonce : Claim → Fp) :`
`∀ c ∈ (systemRollup seed).claims,`
`verifySig K.g K.pub chal c.encode (K.signClaim chal c (nonce c))`

A.23 AiAgentTerraform/Zkp/Project.lean

AiAgentTerraform.Zkp.branchRollup_claims

The claims of a branch's roll-up are the claims of its files.

`theorem branchRollup_claims (seed : Fp) (b : Branch) :`
`(branchRollup seed b).claims = b.files.map claimOfFile`

AiAgentTerraform.Zkp.size_eq_claims_length

The number of steps of a roll-up is the number of claims it carries.

`theorem size_eq_claims_length (t : Rollup) : t.size = t.claims.length`

AiAgentTerraform.Zkp.branchRollup_size

A branch's roll-up has one step per file the branch generates.

`theorem branchRollup_size (seed : Fp) (b : Branch) :`
`(branchRollup seed b).size = b.files.length`

AiAgentTerraform.Zkp.branchesRollup_claims

The claims of the branch roll-up are the claims of all imported files.

`theorem branchesRollup_claims (seed : Fp) :`
`(branchesRollup seed).claims =`
`allBranches.flatMap (fun b => b.files.map claimOfFile)`

AiAgentTerraform.Zkp.branchesRollup_size

The branch roll-up has one step per imported file.

`theorem branchesRollup_size (seed : Fp) :`
`(branchesRollup seed).size = (allBranches.map (fun b => b.files.length)).sum`

AiAgentTerraform.Zkp.branchesRollup_covers

Every file of every imported branch makes a claim in the branch roll-up.

`theorem branchesRollup_covers (seed : Fp) :`
`∀ b ∈ allBranches, ∀ f ∈ b.files,`
`{ claim := claimOfFile f, nonce := nonceOf seed (claimOfFile f) : Attested } ∈`
`(branchesRollup seed).leaves`

AiAgentTerraform.Zkp.projectRollup_claims_eq

The claims of the project are the claims of the system followed by the claims of every imported file.

```
theorem projectRollup_claims_eq (seed : Fp) :  
  (projectRollup seed).claims =  
    (systemRollup seed).claims ++  
      allBranches.flatMap (fun b => b.files.map claimOfFile)
```

AiAgentTerraform.Zkp.projectRollup_size

Nothing is left out and nothing is invented. The project rolls up one step per native step and one per imported file.

```
theorem projectRollup_size (seed : Fp) :  
  (projectRollup seed).size =  
    (systemRollup seed).size + (allBranches.map (fun b => b.files.length)).sum
```

AiAgentTerraform.Zkp.projectRollup_size_eq

The project rolls up 896 steps: the 84 native ones — 60 modelled objects, 16 generated files, 8 operations — and the 812 files of the twenty imported branches.

```
theorem projectRollup_size_eq (seed : Fp) : (projectRollup seed).size = 896
```

AiAgentTerraform.Zkp.projectRollup_covers_system

Every step of the Lean-native system is a step of the project.

```
theorem projectRollup_covers_system (seed : Fp) :  
  ∀ a ∈ (systemRollup seed).leaves, a ∈ (projectRollup seed).leaves
```

AiAgentTerraform.Zkp.projectRollup_covers_branch_files

Every file of every imported branch is a step of the project: the attestation covers the whole repository, not just the modelled part.

```
theorem projectRollup_covers_branch_files (seed : Fp) :  
  ∀ b ∈ allBranches, ∀ f ∈ b.files,  
    { claim := claimOfFile f, nonce := nonceOf seed (claimOfFile f) : Attested } ∈  
      (projectRollup seed).leaves
```

AiAgentTerraform.Zkp.projectRollup_covers_resources

Every modelled resource of the system is a step of the project.

```
theorem projectRollup_covers_resources (seed : Fp) :  
  ∀ p ∈ Tf.fullSystem.parts, ∀ r ∈ p.2.resources,  
    { claim := claimOfResource Tf.Aws.registry r,  
      nonce := nonceOf seed (claimOfResource Tf.Aws.registry r) : Attested } ∈  
      (projectRollup seed).leaves
```

AiAgentTerraform.Zkp.projectRollup_covers_operations

Every operation of the system is a step of the project.

```
theorem projectRollup_covers_operations (seed : Fp) :  
  ∀ o ∈ Ops.operations,  
    { claim := claimOfOp o, nonce := nonceOf seed (claimOfOp o) : Attested } ∈  
      (projectRollup seed).leaves
```

AiAgentTerraform.Zkp.claimOfFile_conforms

The claim of a generated file is type-correct exactly when the file has a path.

```
theorem claimOfFile_conforms (f : GenFile) : (claimOfFile f).conforms = (f.path != "")
```

AiAgentTerraform.Zkp.projectRollup_allConform

Every step of the project is type-correct. Each modelled object conforms to the schema of its type, each operation carries a shebang, and every one of the 812 imported files is generated to a path.

```
theorem projectRollup_allConform (seed : Fp) : (projectRollup seed).allConform = true
```

AiAgentTerraform.Zkp.claimOfFile_costCents

An imported file costs nothing of its own: the resources it describes are costed where they are modelled.

```
theorem claimOfFile_costCents (f : GenFile) : (claimOfFile f).costCents = 0
```

AiAgentTerraform.Zkp.claimOfFile_usage

An imported file uses nothing of its own.

```
theorem claimOfFile_usage (f : GenFile) : (claimOfFile f).usage = Usage.zero
```

AiAgentTerraform.Zkp.sum_eq_zero_of_all

Summing a natural quantity that is identically zero gives zero.

```
private theorem sum_eq_zero_of_all {α : Type*} (g : α → Nat) (l : List α)
  (h : ∀ a ∈ l, g a = 0) : (l.map g).sum = 0
```

AiAgentTerraform.Zkp.usage_sum_eq_zero_of_all

Summing usages that are all zero gives zero usage.

```
private theorem usage_sum_eq_zero_of_all {α : Type*} (g : α → Usage) (l : List α)
  (h : ∀ a ∈ l, g a = Usage.zero) : Usage.sum (l.map g) = Usage.zero
```

AiAgentTerraform.Zkp.branchesRollup_costCents

The imported configuration adds nothing to the bill.

```
theorem branchesRollup_costCents (seed : Fp) : (branchesRollup seed).costCents = 0
```

AiAgentTerraform.Zkp.branchesRollup_usage

The imported configuration uses nothing of its own.

```
theorem branchesRollup_usage (seed : Fp) : (branchesRollup seed).usage = Usage.zero
```

AiAgentTerraform.Zkp.projectRollup_costCents_eq

The project's bill is the system's bill. The 812 imported files add no cost of their own, so what the project attests to is the 3110 cents per month of the modelled infrastructure.

```
theorem projectRollup_costCents_eq (seed : Fp) : (projectRollup seed).costCents = 3110
```

AiAgentTerraform.Zkp.projectRollup_usage_eq

The project attests to the same resource usage as the modelled system: 4 vCPUs, 8 GiB of memory and 60 GiB of storage across 6 instances.

```
theorem projectRollup_usage_eq (seed : Fp) :
  (projectRollup seed).usage = { vcpu := 4, memMiB := 8192, storageGiB := 60, count := 6 }
```

AiAgentTerraform.Zkp.projectCommitment_opens

The project's commitment opens exactly to its total cost, blinded by the sum of the per-step nonces.

```
theorem projectCommitment_opens (P : Params modulus M) (seed : Fp) :
  projectCommitment P seed =
    P.commit (((projectRollup seed).costCents : Fp)) (projectRollup seed).nonceSum
```

AiAgentTerraform.Zkp.projectCommitment_splits

The project's commitment is the sum of the system's and the imported branches'. Each part can publish its own aggregate and the aggregates combine, without anything being reopened.

```
theorem projectCommitment_splits (P : Params modulus M) (seed : Fp) :
  projectCommitment P seed =
    (systemRollup seed).commitTo P Claim.costFp +
    (branchesRollup seed).commitTo P Claim.costFp
```

AiAgentTerraform.Zkp.project_zkpop_complete

Completeness. The project can always prove possession of the opening of the commitment it publishes.

```
theorem project_zkpop_complete (P : Params modulus M) (chal : M → M → Fp) (seed a1 a2 : Fp) :
  Rollup.verify P chal (projectCommitment P seed)
    (Rollup.prove P chal Claim.costFp (projectRollup seed) a1 a2)
```

AiAgentTerraform.Zkp.project_step_proofs_aggregate

The per-step arguments roll up into the project's. Every one of the 896 steps proves possession of the opening of its own commitment; answering a common challenge, those proofs combine into one accepted against the commitment the whole project publishes.

```
theorem project_step_proofs_aggregate (P : Params modulus M) (seed : Fp)
  (rand : Attested → Fp × Fp) (e : Fp) :
  P.verify (projectCommitment P seed)
    (Transcript.combineList e
      ((projectRollup seed).leaves.map (fun a =>
```

Rollup.stepProof P Claim.costFp a (rand a).1 (rand a).2 e)))

AiAgentTerraform.Zkp.project_zkpop_sound

Soundness. Anything that answers two different challenges on the same announcement possesses an opening of the project's published commitment.

```
theorem project_zkpop_sound (P : Params modulus M) (seed : Fp) {T1 T2 : Transcript modulus M}
  (ht : T1.t = T2.t) (he : T1.e ≠ T2.e)
  (h1 : P.verify (projectCommitment P seed) T1) (h2 : P.verify (projectCommitment P seed) T2) :
  P.commit (extract T1 T2).1 (extract T1 T2).2 = projectCommitment P seed
```

AiAgentTerraform.Zkp.project_zkpop_cost_is_declared

The extracted total is the declared total. Under the discrete logarithm assumption the value behind the project's published commitment is exactly the cost its 896 steps attested to.

```
theorem project_zkpop_cost_is_declared (P : Params modulus M) (seed : Fp)
  (hdl : ∀ s : Fp, P.g ≠ s • P.h) {T1 T2 : Transcript modulus M}
  (ht : T1.t = T2.t) (he : T1.e ≠ T2.e)
  (h1 : P.verify (projectCommitment P seed) T1) (h2 : P.verify (projectCommitment P seed) T2) :
  (extract T1 T2).1 = ((projectRollup seed).costCents : Fp)
```

AiAgentTerraform.Zkp.project_zkpop_zero_knowledge

Zero knowledge. The proof the project publishes is exactly what the witness-free simulator produces from the corresponding randomness, so it discloses neither the individual claims nor the total.

```
theorem project_zkpop_zero_knowledge (P : Params modulus M) (seed a1 a2 e : Fp) :
  P.prove ((projectRollup seed).claims.map Claim.costFp).sum (projectRollup seed).nonceSum
    a1 a2 e =
  P.simulate ((projectRollup seed).commitTo P Claim.costFp) e
    (simulateEquiv ((projectRollup seed).claims.map Claim.costFp).sum
      (projectRollup seed).nonceSum e (a1, a2)).1
    (simulateEquiv ((projectRollup seed).claims.map Claim.costFp).sum
      (projectRollup seed).nonceSum e (a1, a2)).2
```

AiAgentTerraform.Zkp.project_steps_signed

Every step of the project carries an operator signature. Signing each of the 896 claims under one key produces signatures that all verify against the single public key.

```
theorem project_steps_signed (K : SigKey modulus M) (chal : M → Fp → Fp) (seed : Fp)
  (nonce : Claim → Fp) :
  ∀ c ∈ (projectRollup seed).claims,
    verifySig K.g K.pub chal c.encode (K.signClaim chal c (nonce c))
```

AiAgentTerraform.Zkp.projectRollup_fits

```
theorem projectRollup_fits (seed : Fp) : (projectRollup seed).size ≤ 2 ^ projectDepth
```

AiAgentTerraform.Zkp.project_step_opens

Every step of the project opens against the published root. Each of the 896 attested steps — modelled object, generated file, operation or imported configuration file — has an authentication path recomputing projectMerkleRoot, revealing nothing about the other steps beyond the digests of the siblings on its path.

```
theorem project_step_opens (seed : Fp) :
  ∀ a ∈ (projectRollup seed).leaves, ∃ p : MPath,
    rootWith digestHash (stepDigest a) p = projectMerkleRoot seed
```

AiAgentTerraform.Zkp.project_open_sound

No step can be invented after the fact. Under injectivity of the compression function, a depth-10 path accepted against the project's root opened one of the project's own step digests (or the padding).

```
theorem project_open_sound (hinj : ∀ a b c e : Fp, digestHash.h a b = digestHash.h c e →
  a = c ∧ b = e)
  (seed : Fp) {v : Fp} {p : MPath} (hp : p.length = projectDepth)
  (h : rootWith digestHash v p = projectMerkleRoot seed) :
  v ∈ (projectRollup seed).stepDigests ∨ v = 0
```

AiAgentTerraform.Zkp.projectOpen_verifies

Each of the 896 steps has a printable opening that verifies against the project's root.

```
theorem projectOpen_verifies (seed : Fp) {i : Nat} (hi : i < 2 ^ projectDepth) :
  rootWith digestHash ((projectRollup seed).padDigest i) (projectOpen seed i) =
    projectMerkleRoot seed
```

AiAgentTerraform.Zkp.merkleRootOfDigests_eq

Digesting the steps once and taking the root of the result is the published root.

```
theorem merkleRootOfDigests_eq (t : Rollup) (d : Nat) :
  merkleRootOfDigests t.stepDigests d = t.merkleRoot d
```

AiAgentTerraform.Zkp.pathOfDigests_eq

Digesting the steps once and computing a path in the result is the published opening.

```
theorem pathOfDigests_eq (t : Rollup) (d i : Nat) :
  pathOfDigests t.stepDigests d i = t.openStep d i
```

AiAgentTerraform.Zkp.demoProjectVerify_true

The project's demonstration always verifies — for every seed and every choice of prover randomness.

```
theorem demoProjectVerify_true (seed a1 a2 : Fp) : demoProjectVerify seed a1 a2 = true
```

A.24 AiAgentTerraform/Zkp/Sigma.lean

AiAgentTerraform.Zkp.Params.smul_commit

Scaling a commitment scales both of its openings.

```
theorem Params.smul_commit (P : Params q M) (c m r : ZMod q) :
  c • P.commit m r = P.commit (c * m) (c * r)
```

AiAgentTerraform.Zkp.Params.verifyB_iff

```
theorem Params.verifyB_iff [DecidableEq M] (P : Params q M) (C : M) (T : Transcript q M) :
  P.verifyB C T = true ↔ P.verify C T
```

AiAgentTerraform.Zkp.Params.completeness

Completeness. The honest prover, who possesses an opening (m, r) of C, always produces an accepting transcript.

```
theorem Params.completeness (P : Params q M) (m r a1 a2 e : ZMod q) :
  P.verify (P.commit m r) (P.prove m r a1 a2 e)
```

AiAgentTerraform.Zkp.Params.special_soundness

Special soundness. Two accepting transcripts with the same announcement and different challenges yield an opening of the statement: the extractor's output is a genuine opening of C, so the prover possessed one.

```
theorem Params.special_soundness (P : Params q M) {C : M} {T1 T2 : Transcript q M}
  (ht : T1.t = T2.t) (he : T1.e ≠ T2.e)
  (h1 : P.verify C T1) (h2 : P.verify C T2) :
  P.commit (extract T1 T2).1 (extract T1 T2).2 = C
```

AiAgentTerraform.Zkp.Params.simulate_verify

Simulated transcripts always verify — the simulator needs no witness.

```
theorem Params.simulate_verify (P : Params q M) (C : M) (e z1 z2 : ZMod q) :
  P.verify C (P.simulate C e z1 z2)
```

AiAgentTerraform.Zkp.Params.honest_verifier_zk

Honest-verifier zero knowledge. For every challenge, the honest transcript produced from randomness (a₁, a₂) is exactly the transcript the witness-free simulator produces from `simulateEquiv` (a₁, a₂). Since `simulateEquiv` is a bijection of the randomness space, the two transcript distributions are identical, and the proof therefore leaks nothing about (m, r).

```
theorem Params.honest_verifier_zk (P : Params q M) (m r a1 a2 e : ZMod q) :
  P.prove m r a1 a2 e =
    P.simulate (P.commit m r) e (simulateEquiv m r e (a1, a2)).1
    (simulateEquiv m r e (a1, a2)).2
```


AiAgentTerraform.Zkp.simulateEquiv_bijective

The randomness map used by zero knowledge is a bijection.

```
theorem simulateEquiv_bijective (m r e : ZMod q) :  
  Function.Bijective (simulateEquiv (q := q) m r e)
```

AiAgentTerraform.Zkp.Params.verify_combine

Aggregation. Two accepting transcripts for the same challenge give an accepting transcript for the sum of their statements.

```
theorem Params.verify_combine (P : Params q M) {C1 C2 : M} {T1 T2 : Transcript q M}  
  (he : T1.e = T2.e) (h1 : P.verify C1 T1) (h2 : P.verify C2 T2) :  
  P.verify (C1 + C2) (Transcript.combine T1 T2)
```

AiAgentTerraform.Zkp.Params.verify_combineList

Aggregation over a list. If every part of the system proves possession of the opening of its own commitment, and all answer the same challenge, the combined transcript proves possession of the opening of the sum of those commitments.

```
theorem Params.verify_combineList (P : Params q M) (e : ZMod q) :  
  ∀ (l : List (M × Transcript q M)),  
    (∀ p ∈ l, p.2.e = e) → (∀ p ∈ l, P.verify p.1 p.2) →  
    P.verify (l.map Prod.fst).sum (Transcript.combineList e (l.map Prod.snd))
```

AiAgentTerraform.Zkp.Params.fsProve_verify

Completeness of the non-interactive proof.

```
theorem Params.fsProve_verify (P : Params q M) (chal : M → M → ZMod q) (m r a1 a2 : ZMod q) :  
  P.fsVerify chal (P.commit m r) (P.fsProve chal m r a1 a2)
```

AiAgentTerraform.Zkp.Params.fsProve_eq_prove

A non-interactive proof is exactly the interactive one at the derived challenge, so soundness and zero knowledge carry over verbatim.

```
theorem Params.fsProve_eq_prove (P : Params q M) (chal : M → M → ZMod q) (m r a1 a2 : ZMod q) :  
  Proof.toTranscript chal (P.commit m r) (P.fsProve chal m r a1 a2) =  
  P.prove m r a1 a2 (chal (P.commit m r) (P.commit a1 a2))
```

A.25 AiAgentTerraform/Zkp/Sign.lean

AiAgentTerraform.Zkp.verifySigB_iff

```
theorem verifySigB_iff [DecidableEq M] (g X : M) (chal : M → ZMod q → ZMod q) (m : ZMod q)  
  (sg : Sig q M) : verifySigB g X chal m sg = true ↔ verifySig g X chal m sg
```

AiAgentTerraform.Zkp.sign_verify

Correctness. A signature made with the secret key always verifies against the public key.

```
theorem sign_verify (K : SigKey q M) (chal : M → ZMod q → ZMod q) (m k : ZMod q) :  
  verifySig K.g K.pub chal m (K.sign chal m k)
```

AiAgentTerraform.Zkp.key_recovery

Nonce reuse recovers the key. Two signatures sharing a nonce commitment but answering different challenges expose the discrete logarithm of the public key: the signer's secret.

```
theorem key_recovery {g X : M} {chal : M → ZMod q → ZMod q} {m1 m2 : ZMod q} {sg1 sg2 : Sig q M}  
  (hR : sg1.R = sg2.R) (he : chal sg1.R m1 ≠ chal sg2.R m2)  
  (h1 : verifySig g X chal m1 sg1) (h2 : verifySig g X chal m2 sg2) :  
  X = ((sg1.s - sg2.s) / (chal sg1.R m1 - chal sg2.R m2)) • g
```

AiAgentTerraform.Zkp.message_binding

A signature is bound to its message. One signature cannot verify for two messages whose challenges differ, unless the public key is the zero element of the group.

```
theorem message_binding {g X : M} {chal : M → ZMod q → ZMod q} {m1 m2 : ZMod q} {sg : Sig q M}  
  (he : chal sg.R m1 ≠ chal sg.R m2)  
  (h1 : verifySig g X chal m1 sg) (h2 : verifySig g X chal m2 sg) : X = 0
```

AiAgentTerraform.Zkp.sign_is_proof_of_knowledge

A signature is a Σ -protocol transcript for possession of the secret key: the announcement is the nonce commitment, the challenge is what the message determines, and the response is the signature. So everything proved about the protocol — soundness of the extractor, the zero-knowledge simulator — applies to signatures verbatim.

```
theorem sign_is_proof_of_knowledge (K : SigKey q M) (chal : M → ZMod q → ZMod q) (m k : ZMod q) :
  (Params.mk K.g (0 : M)).prove K.x 0 k 0 (chal (k • K.g) m) =
    { t := (K.sign chal m k).R, e := chal (k • K.g) m,
      z1 := (K.sign chal m k).s, z2 := 0 }
```

AiAgentTerraform.Zkp.signClaim_verify

A signed claim is accepted against the operator's public key.

```
theorem signClaim_verify (K : SigKey modulus M) (chal : M → Fp → Fp) (c : Claim) (k : Fp) :
  verifySig K.g K.pub chal c.encode (K.signClaim chal c k)
```

AiAgentTerraform.Zkp.signed_rollup_verify

Every step of a roll-up can be signed, and every signature is accepted. Signing each attested claim with its own nonce yields a list of signatures that all verify against the one public key.

```
theorem signed_rollup_verify (K : SigKey modulus M) (chal : M → Fp → Fp) (t : Rollup)
  (nonce : Claim → Fp) :
  ∀ c ∈ t.claims, verifySig K.g K.pub chal c.encode (K.signClaim chal c (nonce c))
```

A.26 AiAgentTerraform/Zkp/System.lean

AiAgentTerraform.Zkp.mem_leaves_node

Membership in a node is membership in one of its children.

```
theorem mem_leaves_node {a : Attested} {t : Rollup} {ks : List Rollup} (l : String)
  (hk : t ∈ ks) (ha : a ∈ t.leaves) : a ∈ (Rollup.node l ks).leaves
```

AiAgentTerraform.Zkp.stackRollup_size

A stack's roll-up has one leaf per modelled object of the stack.

```
theorem stackRollup_size (seed : Fp) (registry : List Tf.ResourceSchema) (path : String)
  (s : Tf.Stack) : (stackRollup seed registry path s).size = s.objectCount
```

AiAgentTerraform.Zkp.systemRollup_size

Every modelled object, every generated file and every operation is a step of the roll-up, and nothing else is.

```
theorem systemRollup_size (seed : Fp) :
  (systemRollup seed).size =
    Tf.fullSystem.objectCount + Tf.fullSystem.files.length + Ops.operations.length
```

AiAgentTerraform.Zkp.systemRollup_covers_resources

Every managed resource of every stack of the system makes a claim.

```
theorem systemRollup_covers_resources (seed : Fp) :
  ∀ p ∈ Tf.fullSystem.parts, ∀ r ∈ p.2.resources,
    { claim := claimOfResource Tf.Aws.registry r,
      nonce := nonceOf seed (claimOfResource Tf.Aws.registry r) : Attested } ∈
    (systemRollup seed).leaves
```

AiAgentTerraform.Zkp.systemRollup_covers_files

Every generated file of the system makes a claim.

```
theorem systemRollup_covers_files (seed : Fp) :
  ∀ f ∈ Tf.fullSystem.files,
    { claim := claimOfFile f, nonce := nonceOf seed (claimOfFile f) : Attested } ∈
    (systemRollup seed).leaves
```

AiAgentTerraform.Zkp.systemRollup_covers_operations

Every operation of the system makes a claim.

```
theorem systemRollup_covers_operations (seed : Fp) :  
  ∀ o ∈ Ops.operations,  
    { claim := claimOfOp o, nonce := nonceOf seed (claimOfOp o) : Attested } ∈  
      (systemRollup seed).leaves
```

AiAgentTerraform.Zkp.systemRollup_claims_eq

The claims of the system, listed directly: for every stack its resources, data sources and module calls, then the generated files, then the operations.

```
theorem systemRollup_claims_eq (seed : Fp) :  
  (systemRollup seed).claims =  
    Tf.fullSystem.parts.flatMap (fun p =>  
      p.2.resources.map (claimOfResource Tf.Aws.registry) ++  
      p.2.dataSources.map (claimOfResource Tf.Aws.registry) ++  
      p.2.modules.map claimOfModule) ++  
    Tf.fullSystem.files.map claimOfFile ++  
    Ops.operations.map claimOfOp
```

AiAgentTerraform.Zkp.systemRollup_allConform

Every step of the system is type-correct. Each modelled object conforms to the schema of its type, each generated file has a path, each operation carries a shebang — so the conformance flag of every claim in the roll-up is set.

```
theorem systemRollup_allConform (seed : Fp) : (systemRollup seed).allConform = true
```

AiAgentTerraform.Zkp.systemRollup_cost_rolls_up

The cost of the system is the sum of the costs of its parts, which is what the roll-up commits to.

```
theorem systemRollup_cost_rolls_up (seed : Fp) :  
  (systemRollup seed).costCents =  
    (Tf.fullSystem.parts.map  
      (fun p => (stackRollup seed Tf.Aws.registry p.1 p.2).costCents)).sum +  
    (Tf.fullSystem.files.map (fun f => (claimOfFile f).costCents)).sum +  
    (Ops.operations.map (fun o => (claimOfOp o).costCents)).sum
```

AiAgentTerraform.Zkp.systemRollup_size_eq

The system rolls up 84 steps: 60 modelled objects, 16 generated files and 8 operations.

```
theorem systemRollup_size_eq (seed : Fp) : (systemRollup seed).size = 84
```

AiAgentTerraform.Zkp.systemRollup_costCents_eq

The cost the system attests to, at the prices of `costTable`: 3110 cents per month.

```
theorem systemRollup_costCents_eq (seed : Fp) : (systemRollup seed).costCents = 3110
```

AiAgentTerraform.Zkp.systemRollup_usage_eq

The resources the system attests to using: 4 vCPUs, 8 GiB of memory and 60 GiB of storage across 6 instances.

```
theorem systemRollup_usage_eq (seed : Fp) :  
  (systemRollup seed).usage = { vcpu := 4, memMiB := 8192, storageGiB := 60, count := 6 }
```

AiAgentTerraform.Zkp.systemCommitment_opens

The commitment opens exactly to the total cost of the system, blinded.

```
theorem systemCommitment_opens (P : Params modulus M) (seed : Fp) :  
  systemCommitment P seed =  
    P.commit (((systemRollup seed).costCents : Fp)) (systemRollup seed).nonceSum
```

AiAgentTerraform.Zkp.system_zkpop_complete

Completeness. The system can always prove possession of the opening of the commitment it publishes.

```
theorem system_zkpop_complete (P : Params modulus M) (chal : M → M → Fp) (seed a1 a2 : Fp) :  
  Rollup.verify P chal (systemCommitment P seed)  
    (Rollup.prove P chal Claim.costFp (systemRollup seed) a1 a2)
```

AiAgentTerraform.Zkp.system_step_proofs_aggregate

The per-step arguments roll up into the system's. Every step of the system — each modelled object, each generated file, each operation — proves possession of the opening of its own commitment; answering a common challenge, those proofs combine into one that is accepted against the commitment the whole system publishes.

```
theorem system_step_proofs_aggregate (P : Params modulus M) (seed : Fp)
  (rand : Attested → Fp × Fp) (e : Fp) :
  P.verify (systemCommitment P seed)
    (Transcript.combineList e
      ((systemRollup seed).leaves.map (fun a =>
        Rollup.stepProof P Claim.costFp a (rand a).1 (rand a).2 e)))
```

AiAgentTerraform.Zkp.system_zkpop_sound

Soundness. Anything that answers two different challenges on the same announcement possesses an opening of the published commitment: the extractor computes one.

```
theorem system_zkpop_sound (P : Params modulus M) (seed : Fp) {T1 T2 : Transcript modulus M}
  (ht : T1.t = T2.t) (he : T1.e ≠ T2.e)
  (h1 : P.verify (systemCommitment P seed) T1) (h2 : P.verify (systemCommitment P seed) T2) :
  P.commit (extract T1 T2).1 (extract T1 T2).2 = systemCommitment P seed
```

AiAgentTerraform.Zkp.system_zkpop_cost_is_declared

The extracted total is the declared total. If the extracted opening claimed a different cost from the sum of the per-step costs, the discrete logarithm of g in base h would follow; so under the discrete-logarithm assumption the value behind the published commitment is exactly the cost the individual resources attested to.

```
theorem system_zkpop_cost_is_declared (P : Params modulus M) (seed : Fp)
  (hdl : ∀ s : Fp, P.g ≠ s • P.h) {T1 T2 : Transcript modulus M}
  (ht : T1.t = T2.t) (he : T1.e ≠ T2.e)
  (h1 : P.verify (systemCommitment P seed) T1) (h2 : P.verify (systemCommitment P seed) T2) :
  (extract T1 T2).1 = ((systemRollup seed).costCents : Fp)
```

AiAgentTerraform.Zkp.system_zkpop_zero_knowledge

Zero knowledge. The proof the system publishes is exactly what the witness-free simulator produces from the corresponding randomness, so it discloses neither the individual claims nor the total.

```
theorem system_zkpop_zero_knowledge (P : Params modulus M) (seed a1 a2 e : Fp) :
  P.prove ((systemRollup seed).claims.map Claim.costFp).sum (systemRollup seed).nonceSum
    a1 a2 e =
  P.simulate ((systemRollup seed).commitTo P Claim.costFp) e
    (simulateEquiv ((systemRollup seed).claims.map Claim.costFp).sum
      (systemRollup seed).nonceSum e (a1, a2)).1
    (simulateEquiv ((systemRollup seed).claims.map Claim.costFp).sum
      (systemRollup seed).nonceSum e (a1, a2)).2
```

AiAgentTerraform.Zkp.demoVerify_true

The demonstration always verifies — for every seed and every choice of prover randomness, not merely for the ones the driver happens to use.

```
theorem demoVerify_true (seed a1 a2 : Fp) : demoVerify seed a1 a2 = true
```